

MẸO

Theo mặc định, TFRecordDataset sẽ đọc từng tệp một, nhưng bạn có thể cho phép nó đọc nhiều tệp song song và xen kẽ các bản ghi bằng cách truyền vào hàm tạo một danh sách đường dẫn tệp và đặt `num_parallel_reads` thành một số lớn hơn một. Ngoài ra, bạn cũng có thể đạt được kết quả tương tự bằng cách sử dụng `list_files()` và `interleave()` như chúng ta đã làm trước đó để đọc nhiều tệp CSV.

Tệp TFRecord đã nén

Việc nén các tệp TFRecord đôi khi rất hữu ích, đặc biệt nếu chúng cần được tải qua kết nối mạng. Bạn có thể tạo tệp TFRecord đã nén bằng cách thiết lập đối số tùy chọn:

```
options = tf.io.TFRecordOptions(compression_type="GZIP") with
tf.io.TFRecordWriter("my_compressed.tfrecord", options) as f:

    f.write(b"Nén, nén, nén!")
```

Khi đọc một tệp TFRecord đã được nén, bạn cần phải chỉ định loại nén:

```
dataset = tf.data.TFRecordDataset(["my_compressed.tfrecord"], compression_type="GZIP")
```

Giới thiệu ngắn gọn về Protocol Buffers

Mặc dù mỗi bản ghi có thể sử dụng bất kỳ định dạng nhị phân nào bạn muốn, nhưng các tệp TFRecord thường chứa các bộ đệm giao thức được tuần tự hóa (còn gọi là protobuf). Đây là một định dạng nhị phân di động, có thể mở rộng và hiệu quả được phát triển tại Google vào năm 2001 và được công khai mã nguồn vào năm 2008; protobuf hiện được sử dụng rộng rãi, đặc biệt là trong [gRPC](#). Hệ thống gọi thủ tục từ xa của Google. Chúng được định nghĩa bằng một ngôn ngữ đơn giản trông như thế này:

```
cú pháp = "proto3";
thông báo Người
    { chuỗi tên = 1; số
      nguyên 32 id = 2;
```

```
    chuỗi email lặp lại = 3;
}
```

Định nghĩa protobuf này cho biết chúng ta đang sử dụng phiên bản 3 của định dạng protobuf, và nó chỉ định rằng mỗi đối tượng ⁴Person có thể (tùy chọn) có tên thuộc loại chuỗi, ID thuộc loại int32, và không hoặc nhiều trường email, mỗi trường thuộc loại chuỗi. Các số 1, 2 và 3 là mã định danh trường: chúng sẽ được sử dụng trong biểu diễn nhị phân của mỗi bản ghi. Sau khi bạn có định nghĩa trong tệp .proto, bạn có thể biên dịch nó. Điều này yêu cầu protoc, trình biên dịch protobuf, để tạo các lớp truy cập trong Python (hoặc một ngôn ngữ khác). Lưu ý rằng các định nghĩa protobuf mà bạn thường sử dụng trong TensorFlow đã được biên dịch sẵn và các lớp Python của chúng là một phần của thư viện TensorFlow, vì vậy bạn sẽ không cần sử dụng protoc. Tất cả những gì bạn cần biết là cách sử dụng các lớp truy cập protobuf trong Python. Để minh họa những điều cơ bản, hãy xem một ví dụ đơn giản sử dụng các lớp truy cập được tạo cho protobuf Person (mã được giải thích trong phần chú thích):

```
>>> from person_pb2 import Person # nhập lớp truy cập được tạo tự động >>> person = Person(name="Al", id=123,
email=["a@b.com"]) # tạo một Person >>> print(person) # hiển thị Person tên: "Al" id: 123 email: "a@b.com" >>>
person.name # đọc trường
'Al' >>> person.name = "Alice" # sửa đổi một trường >>> person.email[0] #
các trường lặp lại
có thể được
truy cập như mảng 'a@b.com'
>>> person.email.append("c@d.com") # thêm địa chỉ email >>>

serialized = person.SerializeToString() # chuyển đổi person thành chuỗi byte >>>
serialized b'\n\x05Alice\x10{\x1a\x07a@b.com\x1a\x07c@d.com' >>> person2 = Person() # tạo một Person mới >>>

person2.ParseFromString(serialized) # phân tích chuỗi byte (dài 27 byte)
```

```
>>> người == người2 # giờ họ bằng nhau
ĐÚNG VẬY
```

Tóm lại, chúng ta nhập lớp Person được tạo bởi protoc, tạo một thể hiện và thao tác với nó, trực quan hóa, đọc và ghi một số trường, sau đó chúng ta tuần tự hóa nó bằng phương thức `SerializeToString()`. Đây là dữ liệu nhị phân sẵn sàng để lưu hoặc truyền qua mạng.

Khi đọc hoặc nhận dữ liệu nhị phân này, chúng ta có thể phân tích nó bằng phương thức `ParseFromString()` và nhận được một bản sao của đối tượng đã được tuần tự hóa.

Chúng ta có thể lưu đối tượng Person đã được tuần tự hóa vào một tệp TFRecord, sau đó chúng ta có thể tải và phân tích cú pháp nó: mọi thứ sẽ hoạt động tốt. Tuy nhiên, `ParseFromString()` không phải là một thao tác của TensorFlow, vì vậy bạn không thể sử dụng nó trong một hàm tiền xử lý trong pipeline `tf.data` (ngoại trừ bằng cách gói nó trong một thao tác `tf.py_function()`, điều này sẽ làm cho mã chậm hơn và kém khả chuyển hơn, như chúng ta đã thấy trong [Chương 12](#)). Tuy nhiên, bạn có thể sử dụng hàm `tf.io.decode_proto()`, hàm này có thể phân tích cú pháp bất kỳ protobuf nào bạn muốn, miễn là bạn cung cấp cho nó định nghĩa protobuf (xem sổ tay để biết ví dụ). Mặc dù vậy, trên thực tế, bạn thường muốn sử dụng các protobuf được định nghĩa trước mà TensorFlow cung cấp các thao tác phân tích cú pháp chuyên dụng. Bây giờ chúng ta hãy xem xét các protobuf được định nghĩa trước này.

TensorFlow Protobufs

Protobuf chính thường được sử dụng trong tệp TFRecord là protobuf Example, đại diện cho một trường hợp trong tập dữ liệu. Nó chứa một danh sách các thuộc tính được đặt tên, trong đó mỗi thuộc tính có thể là một danh sách chuỗi byte, một danh sách số thực hoặc một danh sách số nguyên. Sau đây là định nghĩa protobuf (từ mã nguồn của TensorFlow):

```
cú pháp = "proto3";
message ByteList { giá trị byte lặp lại = 1; } message FloatList
{ giá trị số thực lặp lại = 1 [packed = true]; } message Int64List { giá trị số nguyên
64 lặp lại = 1 [packed = true]; } message Feature { oneof kind {
```

```

        ByteList bytes_list = 1;
        FloatList float_list = 2;
        Int64List int64_list = 3;
    }

}; message Features { map<string, Feature> feature = 1; };
message Example { Features features = 1; };

```

Định nghĩa của ByteList, FloatList và Int64List khá đơn giản. Lưu ý rằng [packed = true] được sử dụng cho các trường số lặp lại, để mã hóa hiệu quả hơn. Một Feature chứa một ByteList, một FloatList hoặc một Int64List. Một Features (với hậu tố s) chứa một từ điển ánh xạ tên tính năng đến giá trị tính năng tương ứng. Và cuối cùng, một Example chỉ chứa một đối tượng Features.

GHI CHÚ

Tại sao lại cần định nghĩa đối tượng Example, trong khi nó chỉ chứa một đối tượng Features? Thực ra, các nhà phát triển của TensorFlow có thể một ngày nào đó sẽ quyết định thêm nhiều trường hơn vào đó. Miễn là định nghĩa Example mới vẫn chứa trường features với cùng ID, thì nó sẽ tương thích ngược. Khả năng mở rộng này là một trong những tính năng tuyệt vời của protobufs.

Dưới đây là cách bạn có thể tạo một tf.train.Example đại diện cho cùng một người như trước đó:

```

from tensorflow.train import ByteList, FloatList, Int64List from
tensorflow.train import Feature, Features, Example

person_example =

    Example( features=Features( feature={ "name":
Feature(bytes_list=ByteList(value= [b"Alice"])), "id":
        Feature(int64_list=Int64List(value=[123])),
"emails": Feature(bytes_list=ByteList(value= [b"a@b.com"],

```

```
b"cd.com"]))) })))
```

Đoạn mã hơi dài dòng và lặp lại, nhưng bạn có thể dễ dàng gói nó vào một hàm trợ giúp nhỏ. Bây giờ chúng ta đã có một protobuf ví dụ, chúng ta có thể tuần tự hóa nó bằng cách gọi phương thức `SerializeToString()` của nó, sau đó ghi dữ liệu kết quả vào một tệp `TFRecord`. Hãy ghi nó năm lần để giả vờ chúng ta có một vài liên hệ:

```
with tf.io.TFRecordWriter("my_contacts.tfrecord") as f:
    for _ in range(5):
        f.write(person_example.SerializeToString())
```

Thông thường bạn sẽ viết nhiều hơn năm ví dụ! Thông thường, bạn sẽ tạo một kịch bản chuyển đổi đọc dữ liệu từ định dạng hiện tại (ví dụ: tệp CSV), tạo một protobuf ví dụ cho mỗi trường hợp, tuần tự hóa chúng và lưu chúng vào một số tệp `TFRecord`, lý tưởng nhất là xáo trộn chúng trong quá trình này. Điều này đòi hỏi một chút công sức, vì vậy một lần nữa hãy chắc chắn rằng nó thực sự cần thiết (có thể quy trình của bạn hoạt động tốt với các tệp CSV).

Giờ chúng ta đã có một tập tin `TFRecord` chứa một số ví dụ được tuần tự hóa, hãy thử tải nó lên.

Ví dụ về tải và phân tích cú pháp

Để tải các protobuf Example đã được tuần tự hóa, chúng ta sẽ sử dụng `tf.data.TFRecordDataset` một lần nữa, và chúng ta sẽ phân tích cú pháp từng Example bằng cách sử dụng `tf.io.parse_single_example()`. Hàm này yêu cầu ít nhất hai đối số: một tensor vô hướng dạng chuỗi chứa dữ liệu đã được tuần tự hóa, và một mô tả cho mỗi thuộc tính. Mô tả là một từ điển ánh xạ tên của mỗi thuộc tính đến một trong hai loại: một mô tả `tf.io.FixedLenFeature` cho biết hình dạng, kiểu và giá trị mặc định của thuộc tính, hoặc một mô tả `tf.io.VarLenFeature` chỉ cho biết kiểu nếu độ dài của danh sách thuộc tính có thể thay đổi (ví dụ như đối với thuộc tính "emails").

Đoạn mã sau định nghĩa một từ điển mô tả, sau đó nó tạo ra một... `TFRecordDataset`, và áp dụng một hàm tiền xử lý tùy chỉnh cho nó, để

Phân tích từng protobuf Example đã được tuần tự hóa mà tập dữ liệu này chứa:

```
mô tả tính năng = {
    "name": tf.io.FixedLenFeature([], tf.string,
default_value=""), "id":
    tf.io.FixedLenFeature([], tf.int64, default_value=0), "emails":
    tf.io.VarLenFeature(tf.string),
}

def parse(serialized_example): return
    tf.io.parse_single_example(serialized_example, feature_description)

dataset =
tf.data.TFRecordDataset(["my_contacts.tfrecord"]).map(parse) for parsed_example
in dataset: print(parsed_example)
```

Các đặc trưng có độ dài cố định được phân tích cú pháp dưới dạng tensor thông thường, nhưng các đặc trưng có độ dài thay đổi được phân tích cú pháp dưới dạng tensor thưa. Bạn có thể chuyển đổi tensor thưa thành tensor dày bằng cách sử dụng `tf.sparse.to_dense()`, nhưng trong trường hợp này, việc truy cập trực tiếp vào giá trị của nó sẽ đơn giản hơn:

```
>>> tf.sparse.to_dense(parsed_example["emails"], default_value=b"")
<tf.Tensor: [...]
dtype=string, numpy=array([b'a@b.com', b'c@d.com'], [...])> >>>
parsed_example["emails"].values
<tf.Tensor: [...] dtype=string,
numpy=array([b'a@b.com', b'c@d.com'], [...])>
```

Thay vì phân tích từng ví dụ một bằng cách sử dụng

`tf.io.parse_single_example()`, bạn có thể muốn phân tích chúng theo từng nhóm bằng cách sử dụng

`tf.io.parse_examples()`:

```
def parse(serialized_examples): return
    tf.io.parse_examples(serialized_examples, feature_description)

dataset =
tf.data.TFRecordDataset(["my_contacts.tfrecord"]).batch(2).map(parse)
```

```
for parsed_examples in dataset:
    print(parsed_examples) # hai ví dụ một lúc
```

Cuối cùng, một `ByteList` có thể chứa bất kỳ dữ liệu nhị phân nào bạn muốn, bao gồm bất kỳ đối tượng được tuần tự hóa nào. Ví dụ, bạn có thể sử dụng `tf.io.encode_jpeg()` để mã hóa một hình ảnh bằng định dạng JPEG và đặt dữ liệu nhị phân này vào một `ByteList`. Sau đó, khi mã của bạn đọc `TFRecord`, nó sẽ bắt đầu bằng cách phân tích `Example`, sau đó nó sẽ cần gọi `tf.io.decode_jpeg()` để phân tích dữ liệu và lấy hình ảnh gốc (hoặc bạn có thể sử dụng `tf.io.decode_image()`, có thể giải mã bất kỳ hình ảnh BMP, GIF, JPEG hoặc PNG nào). Bạn cũng có thể lưu trữ bất kỳ tensor nào bạn muốn trong một `ByteList` bằng cách tuần tự hóa tensor bằng `tf.io.serialize_tensor()` rồi đặt chuỗi byte kết quả vào một thuộc tính `ByteList`. Sau đó, khi bạn phân tích `TFRecord`, bạn có thể phân tích dữ liệu này bằng `tf.io.parse_tensor()`. Xem sổ tay hướng dẫn để biết ví dụ về cách lưu trữ hình ảnh và tensor trong tệp `TFRecord`.

Như bạn thấy, `protobuf Example` khá linh hoạt, vì vậy nó có thể đáp ứng hầu hết các trường hợp sử dụng. Tuy nhiên, nó có thể hơi rắc rối khi bạn làm việc với danh sách các danh sách. Ví dụ, giả sử bạn muốn phân loại tài liệu văn bản. Mỗi tài liệu có thể được biểu diễn dưới dạng một danh sách các câu, trong đó mỗi câu được biểu diễn dưới dạng một danh sách các từ. Và có lẽ mỗi tài liệu cũng có một danh sách các bình luận, trong đó mỗi bình luận được biểu diễn dưới dạng một danh sách các từ. Cũng có thể có một số dữ liệu ngữ cảnh, chẳng hạn như tác giả, tiêu đề và ngày xuất bản của tài liệu. `Protobuf SequenceExample` của TensorFlow được thiết kế cho các trường hợp sử dụng như vậy.

Xử lý danh sách các danh sách bằng cách sử dụng ví dụ `Sequence`.
`Protobuf`

Đây là định nghĩa của `protobuf SequenceExample`:

```
message FeatureList { repeated Feature feature = 1; };
message FeatureLists { map<string, FeatureList> feature_list = 1; };

message SequenceExample {
```

```

    Ngữ cảnh tính năng = 1;
    FeatureLists feature_lists = 2;
};

```

Một `SequenceExample` chứa một đối tượng `Features` cho dữ liệu ngữ cảnh và một đối tượng `FeatureLists` chứa một hoặc nhiều đối tượng `FeatureList` được đặt tên (ví dụ: một `FeatureList` có tên "content" và một `FeatureList` khác có tên "comments"). Mỗi `FeatureList` chứa một danh sách các đối tượng `Feature`, mỗi đối tượng `Feature` có thể là một danh sách các chuỗi byte, một danh sách các số nguyên 64 bit hoặc một danh sách các số thực (trong ví dụ này, mỗi `Feature` sẽ đại diện cho một câu hoặc một bình luận, có thể ở dạng một danh sách các định danh từ). Việc xây dựng một `SequenceExample`, tuần tự hóa nó và phân tích cú pháp nó tương tự như việc xây dựng, tuần tự hóa và phân tích cú pháp một `Example`, nhưng bạn phải sử dụng `tf.io.parse_single_sequence_example()` để phân tích cú pháp một `SequenceExample` duy nhất hoặc `tf.io.parse_sequence_example()` để phân tích cú pháp một lô. Cả hai hàm đều trả về một bộ chứa các đặc trưng ngữ cảnh (dưới dạng từ điển) và các danh sách đặc trưng (cũng dưới dạng từ điển). Nếu danh sách các đặc trưng chứa các chuỗi có kích thước khác nhau (như trong ví dụ trước), bạn có thể muốn chuyển đổi chúng thành các tensor không đều bằng cách sử dụng `tf.RaggedTensor.from_sparse()` (xem notebook để biết mã đầy đủ):

```

parsed_context, parsed_feature_lists =

tf.io.parse_single_sequence_example( serialized_sequence_example,
    context_feature_descriptions,

sequence_feature_descriptions) parsed_content = tf.RaggedTensor.from_sparse(parsed_feature_lists[

```

Giờ bạn đã biết cách lưu trữ, tải, phân tích cú pháp và tiền xử lý dữ liệu hiệu quả bằng API `tf.data`, `TFRecords` và `protobufs`, đã đến lúc chúng ta chuyển sự chú ý sang các lớp tiền xử lý của Keras.

Các lớp tiền xử lý Keras

Việc chuẩn bị dữ liệu cho mạng nơ-ron thường yêu cầu chuẩn hóa các đặc trưng số, mã hóa các đặc trưng phân loại và văn bản, cắt xén và thay đổi kích thước hình ảnh, và nhiều bước khác nữa. Có một số tùy chọn cho việc này:

- Quá trình tiền xử lý có thể được thực hiện trước khi chuẩn bị các tập dữ liệu huấn luyện, sử dụng bất kỳ công cụ nào bạn thích, chẳng hạn như NumPy, Pandas hoặc Scikit-Learn. Bạn cần áp dụng chính xác các bước tiền xử lý tương tự trong môi trường sản xuất để đảm bảo mô hình sản xuất nhận được đầu vào đã được tiền xử lý tương tự như đầu vào đã được sử dụng để huấn luyện.
- Ngoài ra, bạn cũng có thể xử lý trước dữ liệu ngay trong quá trình tải bằng `tf.data`, bằng cách áp dụng một hàm xử lý trước cho từng phần tử của tập dữ liệu bằng phương thức `map()` của tập dữ liệu đó, như chúng ta đã làm trước đó trong chương này. Một lần nữa, bạn sẽ cần áp dụng các bước xử lý trước tương tự trong môi trường sản xuất.
- Một cách tiếp cận cuối cùng là tích hợp trực tiếp các lớp tiền xử lý vào mô hình để nó có thể xử lý tất cả dữ liệu đầu vào ngay trong quá trình huấn luyện, sau đó sử dụng cùng các lớp tiền xử lý đó trong môi trường sản xuất. Phần còn lại của chương này sẽ xem xét cách tiếp cận cuối cùng này.

Keras cung cấp nhiều lớp tiền xử lý mà bạn có thể đưa vào mô hình của mình: chúng có thể được áp dụng cho các đặc trưng số, đặc trưng phân loại, hình ảnh và văn bản. Chúng ta sẽ xem xét các đặc trưng số và phân loại trong các phần tiếp theo, cũng như tiền xử lý văn bản cơ bản, và chúng ta sẽ đề cập đến tiền xử lý hình ảnh trong [Chương 14](#), và tiền xử lý văn bản nâng cao hơn trong [Chương 16](#).

Lớp chuẩn hóa

Như chúng ta đã thấy trong [Chương 10](#), Keras cung cấp một lớp Chuẩn hóa mà bạn có thể sử dụng để chuẩn hóa các đặc trưng đầu vào. Bạn có thể chỉ định giá trị trung bình và phương sai của từng đặc trưng khi tạo lớp, hoặc—đơn giản hơn—bạn có thể truyền tập dữ liệu huấn luyện cho phương thức `adapt()` của lớp.

trước khi huấn luyện mô hình, để lớp đó có thể tự đo lường giá trị trung bình và phương sai của các đặc trưng trước khi huấn luyện:

```
norm_layer = tf.keras.layers.Normalization() model =
tf.keras.models.Sequential([ norm_layer,

    tf.keras.layers.Dense(1)

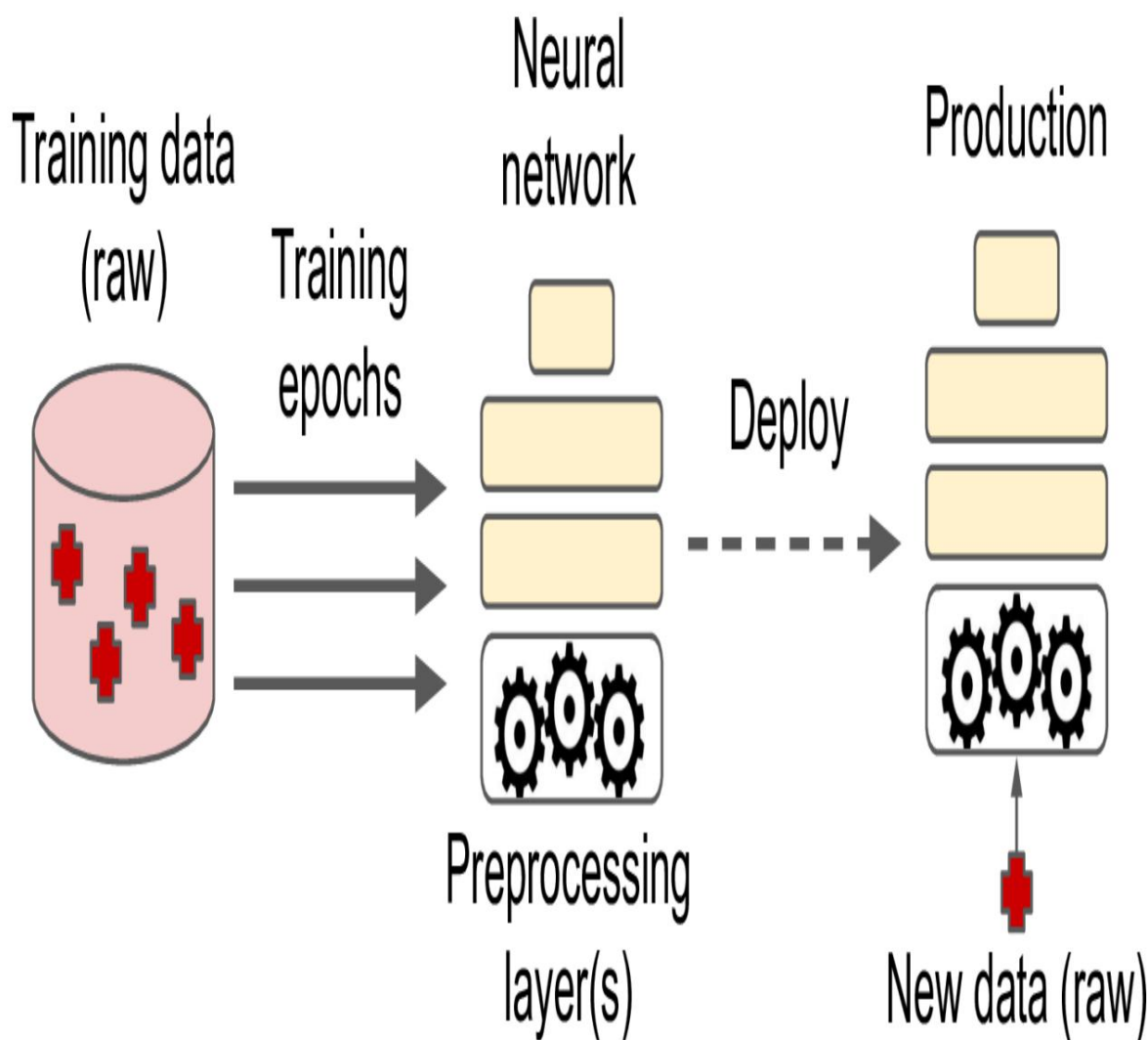
]) model.compile(loss="mse",
optimizer=tf.keras.optimizers.SGD(learning_rate=2e-3))
norm_layer.adapt(X_train) # tính toán giá trị trung bình và phương sai của mỗi
đặc trưng
model.fit(X_train, y_train, validation_data=(X_valid, y_valid), epochs=5)
```

MẸO

Mẫu dữ liệu được truyền vào phương thức `adapt()` phải đủ lớn để đại diện cho tập dữ liệu của bạn, nhưng không nhất thiết phải là toàn bộ tập huấn luyện: đối với lớp Chuẩn hóa, vài trăm trường hợp được lấy mẫu ngẫu nhiên từ tập huấn luyện thường là đủ để có được ước tính tốt về giá trị trung bình và phương sai của các đặc trưng.

Vì chúng ta đã tích hợp lớp Chuẩn hóa vào bên trong mô hình, giờ đây chúng ta có thể triển khai mô hình này lên môi trường sản xuất mà không cần phải lo lắng về việc chuẩn hóa nữa: mô hình sẽ tự động xử lý việc đó (xem [Hình 13-4](#)).

Tuyệt vời! Cách tiếp cận này loại bỏ hoàn toàn nguy cơ sai lệch dữ liệu tiền xử lý: điều này xảy ra khi mọi người cố gắng duy trì mã tiền xử lý khác nhau cho huấn luyện và sản xuất, nhưng họ cập nhật một mã mà quên cập nhật mã còn lại. Mô hình sản xuất cuối cùng nhận được dữ liệu được tiền xử lý theo cách mà nó không mong đợi. Nếu may mắn, họ sẽ phát hiện ra một lỗi rõ ràng. Nếu không, độ chính xác của mô hình sẽ âm thầm giảm xuống.



Hình 13-4. Bao gồm các lớp tiền xử lý bên trong một mô hình.

Việc đưa trực tiếp lớp tiền xử lý vào mô hình rất hay và đơn giản, nhưng nó sẽ làm chậm quá trình huấn luyện (chỉ một chút trong trường hợp lớp Chuẩn hóa): thực tế, vì quá trình tiền xử lý được thực hiện ngay trong quá trình huấn luyện, nên nó diễn ra một lần mỗi epoch. Chúng ta có thể làm tốt hơn bằng cách chuẩn hóa toàn bộ tập dữ liệu huấn luyện chỉ một lần trước khi huấn luyện. Để làm điều này, bạn có thể sử dụng lớp Chuẩn hóa một cách độc lập (tương tự như StandardScaler của Scikit-Learn):

```
norm_layer = tf.keras.layers.Normalization()
norm_layer.adapt(X_train)
X_train_scaled = norm_layer(X_train)
X_valid_scaled = norm_layer(X_valid)
```

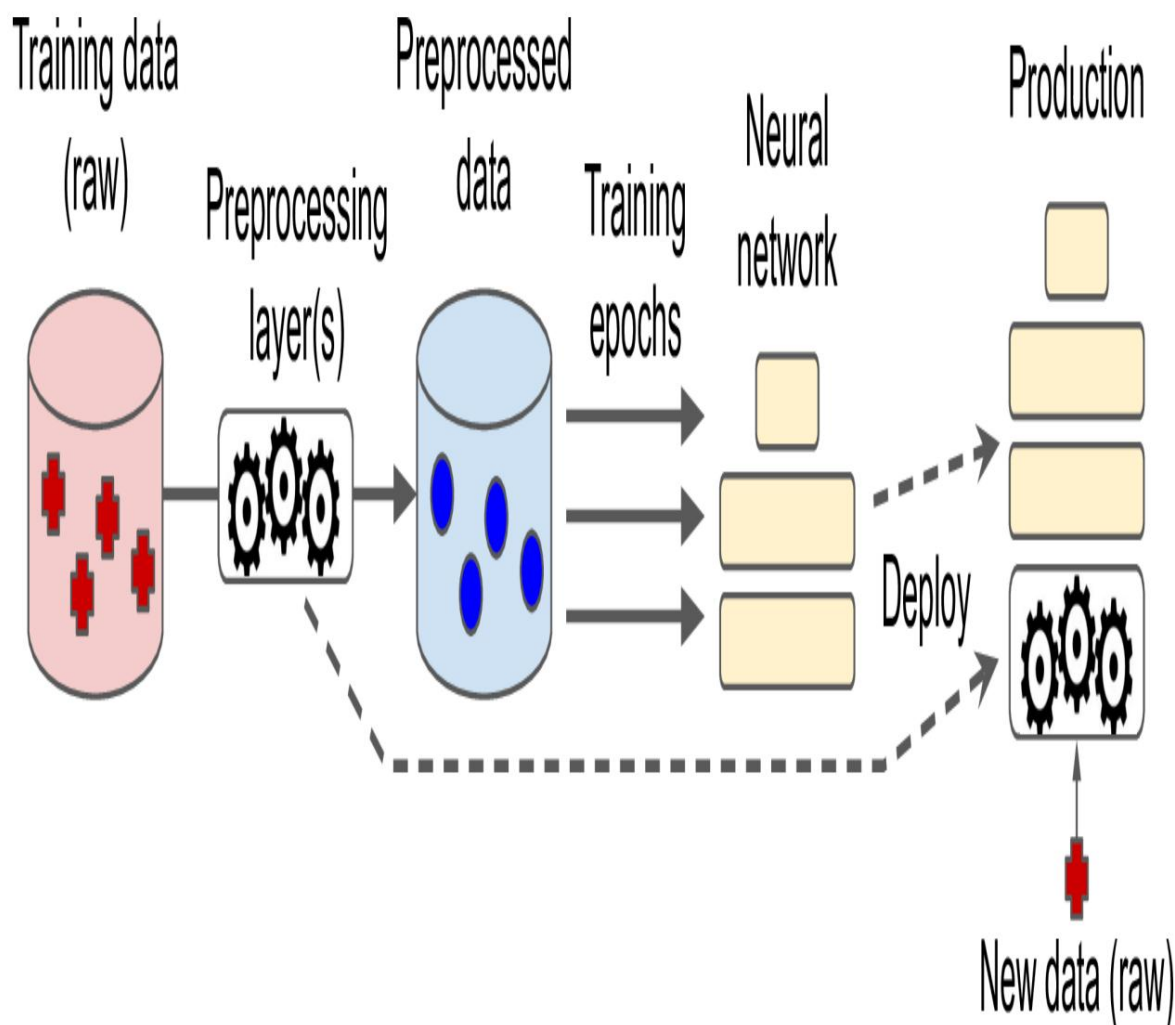
Giờ đây chúng ta có thể huấn luyện một mô hình trên dữ liệu đã được hiệu chỉnh, nhưng lần này không cần bất kỳ điều kiện nào khác.

Lớp chuẩn hóa bên trong mô hình:

```
model = tf.keras.models.Sequential([tf.keras.layers.Dense(1)])
model.compile(loss="mse",
optimizer=tf.keras.optimizers.SGD(learning_rate=2e-3))
model.fit(X_train_scaled, y_train, epochs=5,
validation_data=(X_valid_scaled, y_valid))
```

Tuyệt vời! Điều này sẽ giúp tăng tốc quá trình huấn luyện một chút. Nhưng giờ đây mô hình sẽ không xử lý trước dữ liệu đầu vào khi chúng ta triển khai nó lên môi trường sản xuất. Để khắc phục điều này, chúng ta chỉ cần tạo một mô hình mới bao gồm cả lớp Chuẩn hóa đã được điều chỉnh và mô hình mà chúng ta vừa huấn luyện. Sau đó, chúng ta có thể triển khai mô hình cuối cùng này lên môi trường sản xuất, và nó sẽ tự động xử lý trước dữ liệu đầu vào và đưa ra dự đoán (xem [Hình 13-5](#)):

```
final_model = tf.keras.Sequential([norm_layer, model])
X_new = X_test[:3] # giả vờ chúng ta có một vài trường hợp mới
(chưa được
chia tỷ lệ) y_pred = final_model(X_new) # xử lý trước dữ liệu và đưa
ra dự đoán
```



Hình 13-5. Xử lý sơ bộ dữ liệu chỉ một lần trước khi huấn luyện bằng cách sử dụng các lớp xử lý sơ bộ, sau đó triển khai các lớp này bên trong mô hình cuối cùng.

Giờ đây chúng ta có được những ưu điểm của cả hai phương pháp: quá trình huấn luyện diễn ra nhanh chóng vì chúng ta chỉ cần xử lý sơ bộ dữ liệu một lần trước khi bắt đầu huấn luyện, và mô hình cuối cùng có thể xử lý sơ bộ đầu vào của nó ngay lập tức mà không có nguy cơ xảy ra sự không khớp trong quá trình xử lý sơ bộ.

Hơn nữa, các lớp tiền xử lý của Keras hoạt động rất tốt với API `tf.data`. Ví dụ, có thể truyền một `tf.data.Dataset` cho phương thức `adapt()` của lớp tiền xử lý. Cũng có thể áp dụng một lớp tiền xử lý Keras cho một `tf.data.Dataset` bằng cách sử dụng phương thức `map()` của tập dữ liệu. Ví dụ, đây là cách bạn có thể áp dụng một lớp Chuẩn hóa đã được điều chỉnh cho các đặc trưng đầu vào của mỗi lô trong một tập dữ liệu:

```
dataset = dataset.map(lambda X, y: (norm_layer(X), y))
```

Cuối cùng, nếu bạn cần nhiều tính năng hơn những gì các lớp tiền xử lý của Keras cung cấp, bạn luôn có thể tự viết lớp Keras của riêng mình, giống như chúng ta đã thảo luận trong **Chương 12**. Ví dụ, nếu lớp Chuẩn hóa không tồn tại, bạn có thể nhận được kết quả tương tự bằng cách sử dụng lớp tùy chỉnh sau:

```
import numpy as np

class MyNormalization(tf.keras.layers.Layer): def
    adapt(self, X):
        self.mean_ = np.mean(X, axis=0, keepdims=True) self.std_ =
        np.std(X, axis=0, keepdims=True)

    def call(self, inputs): eps =
        tf.keras.backend.epsilon() # một phép làm mịn nhỏ
        thuật ngữ
        trả về (đầu vào - self.mean_) / (self.std_ + eps)
```

Tiếp theo, chúng ta hãy xem xét một lớp tiền xử lý khác của Keras dành cho các đặc trưng số: lớp Rời rạc hóa (Discretization layer).

Lớp rời rạc

Mục tiêu của lớp Rời rạc hóa là chuyển đổi một đặc trưng số thành một đặc trưng phân loại bằng cách ánh xạ các khoảng giá trị (gọi là bin) đến các danh mục. Điều này đôi khi hữu ích cho các đặc trưng có phân bố đa đỉnh, hoặc với các đặc trưng có mối quan hệ phi tuyến tính cao với mục tiêu. Ví dụ, đoạn mã sau ánh xạ một đặc trưng tuổi số thành ba danh mục: dưới 18, từ 18 đến 50 (không bao gồm), và từ 50 trở lên.

```
>>> age = tf.constant([[10.], [93.], [57.], [18.], [37.], [5.]]) >>> discretize_layer
=
tf.keras.layers.Discretization(bin_boundaries=[18., 50.]) >>> age_categories
= discretize_layer(age) >>> age_categories <tf.Tensor:
shape=(6, 1),
dtype=int64, numpy=array([[0],[2],[2], [1],[1],[0]])>
```

Trong ví dụ này, chúng ta đã cung cấp các ranh giới thùng mong muốn. Nếu muốn, bạn cũng có thể cung cấp số lượng thùng bạn muốn, sau đó gọi lớp đó.

Phương thức `adapt()` được sử dụng để tìm các ranh giới nhóm phù hợp, dựa trên các phân vị giá trị. Ví dụ, nếu ta đặt `num_bins=3`, thì các ranh giới nhóm sẽ nằm ở các giá trị ngay dưới phân vị thứ 33 và 66 (trong ví dụ này, ở các giá trị 10 và 37).

```
>>> discretize_layer = tf.keras.layers.Discretization(num_bins=3) >>>
discretize_layer.adapt(age) >>>
age_categories = discretize_layer(age) >>> age_categories
<tf.Tensor: shape=(6, 1),
dtype=int64, numpy=array([[1],[2],[2], [1],[2],[0]])>
```

Các định danh danh mục như thế này thường không nên được truyền trực tiếp vào mạng nơ-ron, vì giá trị của chúng không thể được so sánh một cách có ý nghĩa. Thay vào đó, chúng nên được mã hóa, ví dụ, bằng cách sử dụng mã hóa one-hot. Bây giờ chúng ta hãy xem cách thực hiện điều này.

Lớp mã hóa danh mục

Khi chỉ có một vài danh mục (ví dụ: ít hơn một hoặc hai chục), thì mã hóa one-hot thường là một lựa chọn tốt (như đã thảo luận trong [Chương 2](#)). Để làm điều này, Keras cung cấp lớp `CategoryEncoding`. Ví dụ, hãy mã hóa one-hot cho thuộc tính `age_categories` mà chúng ta vừa tạo:

```
>>> onehot_layer = tf.keras.layers.CategoryEncoding(num_tokens=3) >>>
onehot_layer(age_categories) <tf.Tensor:
shape=(6, 3), dtype=float32, numpy= array([[0., 1., 0.], [0., 0.,
1.], [0., 0., 1.], [0., 1.,
0.], [0., 0.,
1.], [1., 0.,
0.]],
dtype=float32)>
```

Nếu bạn cố gắng mã hóa nhiều hơn một đặc trưng phân loại cùng một lúc (điều này chỉ có ý nghĩa nếu tất cả chúng đều sử dụng cùng một danh mục), thì...

Lớp `CategoryEncoding` sẽ thực hiện mã hóa đa điểm nóng theo mặc định:

Tensor đầu ra sẽ chứa giá trị 1 cho mỗi danh mục có trong bất kỳ đặc trưng đầu vào nào. Ví dụ:

```
>>> two_age_categories = np.array([[1, 0], [2, 2], [2, 0]]) >>>
onehot_layer(two_age_categories) <tf.Tensor:
shape=(3, 3), dtype=float32, numpy= array([[1., 1., 0.], [0.,
0., 1.], [1., 0., 1.]],
dtype=float32)>
```

Nếu bạn cho rằng việc biết mỗi danh mục xuất hiện bao nhiêu lần là hữu ích, bạn có thể đặt `output_mode="count"` khi tạo lớp `CategoryEncoding`, trong trường hợp đó tensor đầu ra sẽ chứa số lần xuất hiện của mỗi danh mục. Trong ví dụ trên, đầu ra sẽ giống nhau ngoại trừ hàng thứ hai, sẽ trở thành `[0., 0., 2.]`.

Lưu ý rằng cả mã hóa đa điểm nóng (multi-hot encoding) và mã hóa đếm (count encoding) đều làm mất thông tin, vì không thể biết mỗi danh mục đang hoạt động đến từ đặc trưng nào. Ví dụ, cả `[0, 1]` và `[1, 0]` đều được mã hóa thành `[1., 1., 0.]`. Nếu bạn muốn tránh điều này, bạn cần mã hóa một điểm nóng (one-hot encoding) cho từng đặc trưng riêng biệt và nối các đầu ra lại với nhau. Bằng cách này, `[0, 1]` sẽ được mã hóa thành `[1., 0., 0., 0., 1., 0.]` và `[1, 0]` sẽ được mã hóa thành `[0., 1., 0., 1., 0., 0.]`. Bạn có thể nhận được kết quả tương tự bằng cách điều chỉnh các định danh danh mục sao cho chúng không trùng lặp, ví dụ:

```
>>> onehot_layer = tf.keras.layers.CategoryEncoding(num_tokens=3 + 3) >>>

onehot_layer(two_age_categories + [0, 3]) # thêm 3 vào đặc trưng thứ hai <tf.Tensor:
shape=(3, 6),
dtype=float32, numpy= array([[0., 1., 0., 1., 0., 0.], [0., 0.,
1., 0., 0., 1.], [0., 0., 1., 1., 0., 0.]],
dtype=float32)>
```

Trong kết quả này, ba cột đầu tiên tương ứng với đặc trưng thứ nhất, và ba cột cuối cùng tương ứng với đặc trưng thứ hai. Điều này cho phép mô hình phân biệt hai đặc trưng. Tuy nhiên, nó cũng làm tăng số lượng...

Các đặc trưng được đưa vào mô hình, và do đó yêu cầu nhiều tham số mô hình hơn. Thật khó để biết trước liệu mã hóa đa điểm nóng đơn lẻ hay mã hóa một điểm nóng cho từng đặc trưng sẽ hoạt động tốt nhất: điều đó phụ thuộc vào nhiệm vụ, và bạn có thể cần phải thử cả hai tùy chọn.

Giờ đây, bạn có thể mã hóa các đặc trưng số nguyên phân loại bằng cách sử dụng mã hóa one-hot hoặc multi-hot. Nhưng còn các đặc trưng văn bản phân loại thì sao? Đối với điều này, bạn có thể sử dụng lớp StringLookup.

Lớp StringLookup

Bạn cũng có thể sử dụng lớp StringLookup của Keras để mã hóa one-hot một thuộc tính của các thành phố:

```
>>> cities = ["Auckland", "Paris", "Paris", "San Francisco"] >>>
str_lookup_layer = tf.keras.layers.StringLookup() >>>
str_lookup_layer.adapt(cities) >>>
str_lookup_layer(["Paris"], ["Auckland"], ["Auckland"],
["Montreal"]])
<tf.Tensor: shape=(4, 1), dtype=int64, numpy=array([[1], [3],
[3], [0]])>
```

Đầu tiên, chúng ta tạo một lớp StringLookup, sau đó điều chỉnh nó cho phù hợp với dữ liệu: nó tìm ra rằng có ba danh mục khác nhau. Sau đó, chúng ta sử dụng lớp này để mã hóa một vài thành phố: chúng được mã hóa dưới dạng số nguyên theo mặc định. Các danh mục không xác định được gán giá trị 0, như trường hợp của "Montreal" trong ví dụ này. Các danh mục đã biết được đánh số bắt đầu từ 1, từ danh mục xuất hiện nhiều nhất đến danh mục xuất hiện ít nhất.

Thuận tiện hơn, nếu bạn đặt `output_mode="one_hot"` khi tạo

Lớp StringLookup sẽ xuất ra một vectơ one-hot cho mỗi danh mục, thay vì một số nguyên:

```
>>> str_lookup_layer =
tf.keras.layers.StringLookup(output_mode="one_hot")
>>> str_lookup_layer.adapt(cities)
>>> str_lookup_layer(["Paris"], ["Auckland"], ["Auckland"],
["Montreal"]])
<tf.Tensor: shape=(4, 4), dtype=float32, numpy=
```

```
array([[0., 1., 0., 0.], [0.,
        0., 0., 1.], [0., 0.,
        0., 1.], [1., 0., 0.,
        0.]], dtype=float32)>
```

MẸO

Keras cũng bao gồm một lớp IntegerLookup hoạt động tương tự như lớp StringLookup nhưng nhận đầu vào là số nguyên thay vì chuỗi ký tự.

Nếu tập dữ liệu huấn luyện rất lớn, việc điều chỉnh lớp chỉ trên một tập con ngẫu nhiên của tập dữ liệu huấn luyện có thể là lựa chọn phù hợp. Trong trường hợp này, phương thức `adapt()` của lớp có thể bỏ sót một số danh mục hiếm gặp. Theo mặc định, nó sẽ ánh xạ tất cả chúng vào danh mục 0, khiến mô hình không thể phân biệt được chúng. Để giảm thiểu rủi ro này (trong khi vẫn chỉ điều chỉnh lớp trên một tập con của tập dữ liệu huấn luyện), bạn có thể đặt `num_oov_indices` thành một số nguyên lớn hơn 1. Đây là số lượng các nhóm từ vựng không có trong từ điển (OOV) cần sử dụng: mỗi danh mục không xác định sẽ được gán ngẫu nhiên vào một trong các nhóm OOV, sử dụng hàm băm lấy dư theo số lượng nhóm OOV. Điều này sẽ cho phép mô hình phân biệt được ít nhất một số danh mục hiếm. Ví dụ:

```
>>> str_lookup_layer =
tf.keras.layers.StringLookup(num_oov_indices=5) >>>
str_lookup_layer.adapt(cities) >>>
str_lookup_layer(["Paris"], ["Auckland"], ["Foo"], ["Bar"], ["Baz"]]) <tf.Tensor:
shape=(4,
1), dtype=int64, numpy=array([[5], [7], [4], [3], [4]])>
```

Vì có 5 nhóm Oov, nên ID của danh mục đầu tiên được biết đến hiện là 5 ("Paris"). Nhưng "Foo", "Bar" và "Baz" chưa được biết đến, vì vậy mỗi danh mục được gán vào một trong các nhóm Oov. "Bar" được gán vào nhóm riêng (với ID 3), nhưng thật không may, "Foo" và "Baz" lại được gán vào cùng một nhóm (với ID 4), do đó mô hình không thể phân biệt chúng. Điều này được gọi là xung đột băm. Cách duy nhất để giảm nguy cơ xung đột là...

Tăng số lượng nhóm OOVA. Tuy nhiên, điều này cũng sẽ làm tăng tổng số lượng danh mục, dẫn đến cần nhiều RAM hơn và thêm các tham số mô hình sau khi các danh mục được mã hóa one-hot. Vì vậy, đừng tăng số lượng này quá nhiều.

Ý tưởng về việc gán các danh mục một cách ngẫu nhiên vào các nhóm được gọi là thủ thuật băm (hashing trick). Keras cung cấp một lớp chuyên dụng thực hiện chính điều đó: lớp Băm (Hashing layer).

Lớp băm

Đối với mỗi danh mục, lớp băm Keras tính toán một giá trị băm, lấy phần dư của số lượng nhóm (hoặc "bin"). Việc ánh xạ hoàn toàn là ngẫu nhiên giả, nhưng ổn định trên các lần chạy và nền tảng khác nhau (nghĩa là cùng một danh mục sẽ luôn được ánh xạ tới cùng một số nguyên, miễn là số lượng bin không thay đổi).

Ví dụ, chúng ta hãy sử dụng lớp Hashing để mã hóa một vài thành phố:

```
>>> hashing_layer = tf.keras.layers.Hashing(num_bins=10) >>>
hashing_layer([["Paris"], ["Tokyo"], ["Auckland"],
["Montreal"]])
<tf.Tensor: shape=(4, 1), dtype=int64, numpy=array([[0], [1],
[9], [1]])>
```

Ưu điểm của lớp này là nó không cần phải được điều chỉnh gì cả, điều này đôi khi rất hữu ích, đặc biệt là trong trường hợp dữ liệu không nằm trong bộ nhớ chính (khi tập dữ liệu quá lớn để chứa vừa trong bộ nhớ). Tuy nhiên, chúng ta lại gặp phải xung đột băm: "Tokyo" và "Montreal" được ánh xạ tới cùng một ID, khiến mô hình không thể phân biệt được chúng. Vì vậy, thông thường nên sử dụng lớp StringLookup.

Bây giờ chúng ta hãy xem xét một cách khác để mã hóa các danh mục: nhưng có thể huấn luyện được.

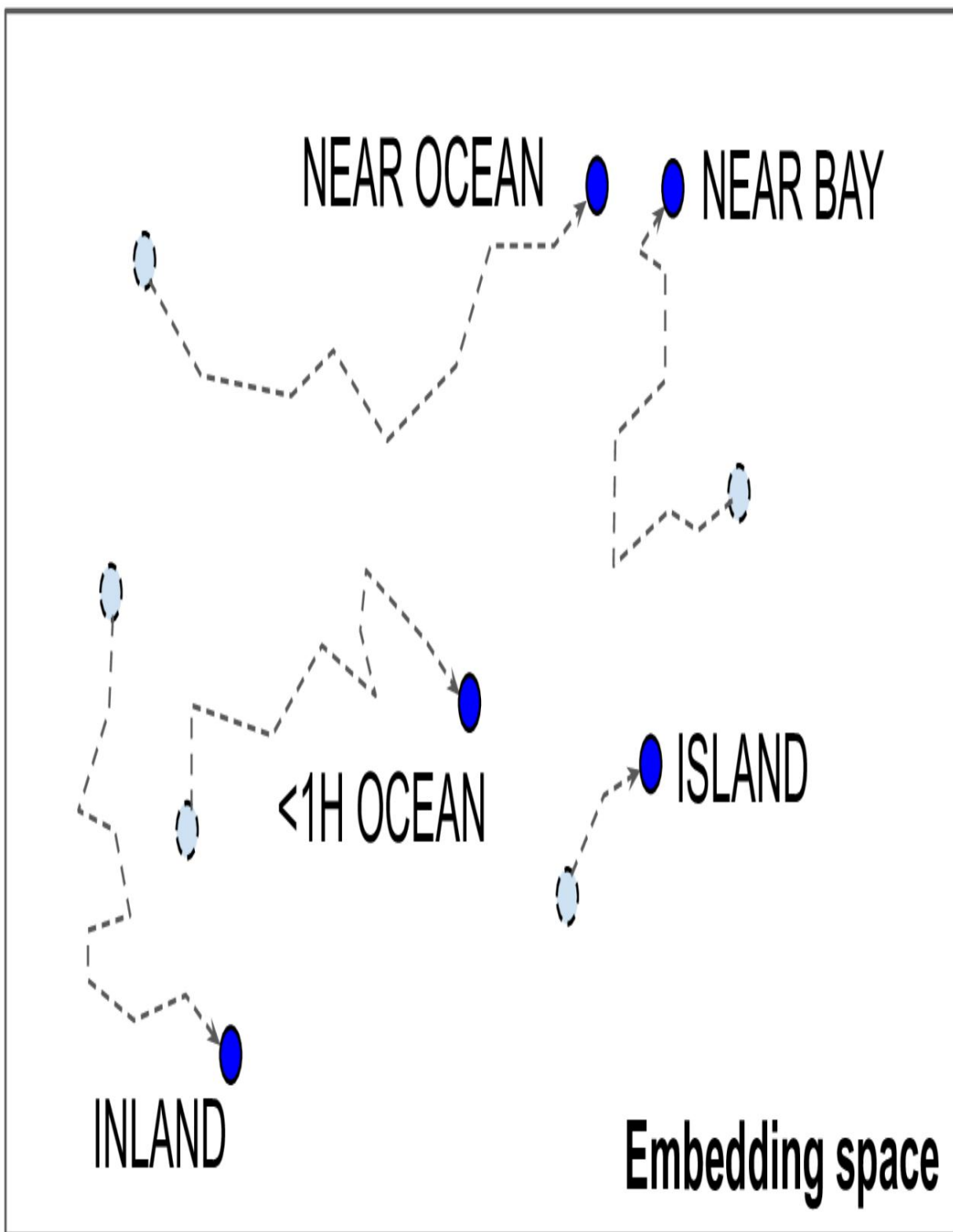
Mã hóa các đặc trưng phân loại bằng cách sử dụng phép nhúng

Một embedding là một biểu diễn cô đọng của dữ liệu đa chiều, chẳng hạn như một danh mục hoặc một từ trong từ vựng. Nếu có 50.000 danh mục khả thi, thì mã hóa one-hot sẽ tạo ra một không gian 50.000 chiều.

Vector thưa (tức là chủ yếu chứa số 0). Ngược lại, một embedding sẽ là một vector dày đặc tương đối nhỏ, ví dụ chỉ có 100 chiều.

Trong học sâu (Deep Learning), các embedding thường được khởi tạo ngẫu nhiên, sau đó được huấn luyện bằng thuật toán Gradient Descent cùng với các tham số khác của mô hình. Ví dụ, danh mục "GẦN VỊNH" trong tập dữ liệu nhà ở California ban đầu có thể được biểu diễn bằng một vectơ ngẫu nhiên như $[0.131, 0.890]$, trong khi danh mục "GẦN ĐẠI DƯƠNG" có thể được biểu diễn bằng một vectơ ngẫu nhiên khác như $[0.631, 0.791]$. Trong ví dụ này, chúng ta sử dụng embedding 2D, nhưng số chiều là một siêu tham số mà bạn có thể điều chỉnh.

Vì các embedding này có thể huấn luyện được, chúng sẽ dần được cải thiện trong quá trình huấn luyện; và vì chúng đại diện cho các danh mục khá giống nhau trong trường hợp này, thuật toán Gradient Descent chắc chắn sẽ đẩy chúng lại gần nhau hơn, đồng thời có xu hướng đẩy chúng ra xa embedding của danh mục "INLAND" (xem [Hình 13-6](#)). Thật vậy, biểu diễn càng tốt thì mạng nơ-ron càng dễ đưa ra dự đoán chính xác, vì vậy quá trình huấn luyện có xu hướng làm cho các embedding trở thành những biểu diễn hữu ích của các danh mục. Điều này được gọi là học biểu diễn (chúng ta sẽ thấy các loại học biểu diễn khác trong [Chương 17](#)).



Hình 13-6. Các embedding sẽ dần được cải thiện trong quá trình huấn luyện.

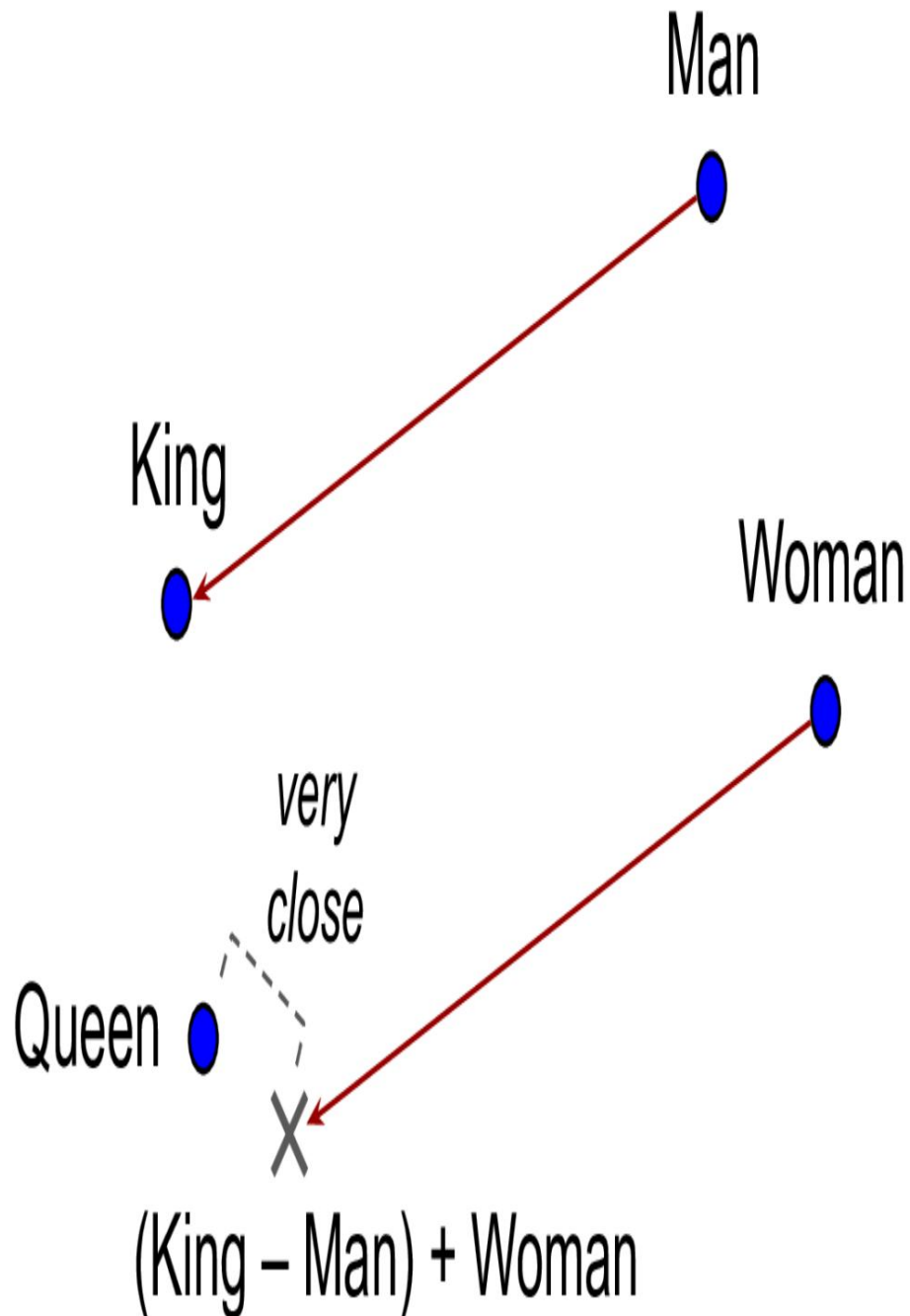
NHÚNG TỪ

Không chỉ các embedding nói chung là những biểu diễn hữu ích cho nhiệm vụ đang thực hiện, mà chúng thường có thể được tái sử dụng thành công cho các nhiệm vụ khác. Ví dụ phổ biến nhất về điều này là các embedding từ (tức là embedding của từng từ riêng lẻ): khi bạn đang thực hiện một nhiệm vụ xử lý ngôn ngữ tự nhiên, bạn thường sẽ có lợi hơn nếu tái sử dụng các embedding từ đã được huấn luyện trước thay vì tự huấn luyện.

Ý tưởng sử dụng vectơ để biểu diễn từ ngữ đã có từ những năm 1960, và nhiều kỹ thuật phức tạp đã được sử dụng để tạo ra các vectơ hữu ích, bao gồm cả việc sử dụng mạng nơ-ron. Nhưng mọi thứ thực sự bùng nổ vào năm 2013, khi Tomáš Mikolov và các nhà nghiên cứu khác của Google công bố một ⁶[bài báo](#). Bài viết mô tả một kỹ thuật hiệu quả để học nhúng từ bằng mạng nơ-ron, vượt trội đáng kể so với các phương pháp trước đó. Điều này cho phép họ học cách tạo các embedding trên một kho văn bản rất lớn: họ đã huấn luyện một mạng nơ-ron để dự đoán các từ gần bất kỳ từ nào được cho trước, và thu được các embedding từ đáng kinh ngạc. Ví dụ, các từ đồng nghĩa có embedding rất gần nhau, và các từ có liên quan về mặt ngữ nghĩa như Pháp, Tây Ban Nha và Ý cuối cùng được nhóm lại với nhau.

Tuy nhiên, không chỉ đơn thuần là vấn đề về sự gần gũi: các vectơ nhúng từ cũng được sắp xếp dọc theo các trục có ý nghĩa trong không gian nhúng. Đây là một ví dụ nổi tiếng: nếu bạn tính toán King - Man + Woman (cộng và trừ các vectơ nhúng của các từ này), thì kết quả sẽ rất gần với vectơ nhúng của từ Queen (xem [Hình 13-7](#)). Nói cách khác, các vectơ nhúng từ mã hóa khái niệm về giới tính! Tương tự, bạn có thể tính Madrid - Tây Ban Nha + Pháp, và kết quả gần với Paris, điều này dường như cho thấy khái niệm thủ đô cũng được mã hóa trong các embedding.

Embedding space



Hình 13-7. Các biểu diễn từ của những từ tương tự có xu hướng gần nhau, và một số trục đường như mã hóa các khái niệm có ý nghĩa.

Thật không may, đôi khi các phép nhúng từ lại phản ánh những định kiến tồi tệ nhất của chúng ta. Ví dụ, mặc dù họ học đúng rằng đàn ông đối với vua cũng như phụ nữ đối với hoàng hậu, nhưng dường như họ cũng học rằng đàn ông đối với bác sĩ cũng như phụ nữ đối với y tá: một sự thiên vị giới tính khá rõ rệt! Công bằng mà nói, ví dụ cụ thể này có lẽ đã bị phóng đại, như đã được chỉ ra trong một **bài báo năm 2019**.⁷ (bởi Malvina Nissim và cộng sự). Tuy nhiên, đảm bảo tính công bằng trong các thuật toán Học sâu là một chủ đề nghiên cứu quan trọng và đang được tích cực quan tâm.

Keras cung cấp một lớp Embedding, bao bọc một ma trận nhúng: ma trận này có một hàng cho mỗi danh mục và một cột cho mỗi chiều nhúng. Theo mặc định, nó được khởi tạo ngẫu nhiên. Để chuyển đổi ID danh mục thành một embedding, lớp Embedding chỉ cần tìm kiếm và trả về hàng tương ứng với danh mục đó. Chỉ đơn giản vậy thôi! Ví dụ, hãy khởi tạo một lớp Embedding với 5 hàng và các embedding 2D, và hãy sử dụng nó để mã hóa một số danh mục:

```
>>> tf.random.set_seed(42) >>>
embedding_layer = tf.keras.layers.Embedding(input_dim=5, output_dim=2)
>>>
embedding_layer(np.array([2, 4, 2])) <tf.Tensor:
shape=(3, 2), dtype=float32, numpy= array([[ -0.04663396,
 0.01846724], [ -0.02736737, -0.02768031],
        [ -0.04663396, 0.01846724]]),
dtype=float32)>
```

Như bạn thấy, hạng mục 2 được mã hóa (hai lần) thành vectơ 2D [-0.04663396, 0.01846724], trong khi hạng mục 4 được mã hóa thành [-0.02736737, -0.02768031]. Vì lớp này chưa được huấn luyện, nên các mã hóa này chỉ là ngẫu nhiên.

CẢNH BÁO

Lớp Embedding được khởi tạo ngẫu nhiên, vì vậy việc sử dụng nó bên ngoài mô hình như một lớp tiền xử lý độc lập là không hợp lý, trừ khi bạn khởi tạo nó với các trọng số đã được huấn luyện trước.

Nếu bạn muốn nhúng thuộc tính văn bản phân loại, bạn chỉ cần nối chuỗi một Lớp StringLookup và lớp Embedding, như sau:

```
>>> tf.random.set_seed(42)
>>> ocean_prox = ["<1H OCEAN", "INLAND", "NEAR OCEAN", "NEAR
BAY", "ISLAND"]
>>> str_lookup_layer = tf.keras.layers.StringLookup()
>>> str_lookup_layer.adapt(ocean_prox)
>>> lookup_and_embed = tf.keras.Sequential([
...     str_lookup_layer,
...     tf.keras.layers.Embedding(input_dim=str_lookup_layer.vocabulary_size
...     ),
...     tf.keras.layers.Dense(output_dim=2)
... ])
>>> lookup_and_embed(np.array([["<1H OCEAN"], ["ISLAND"], ["<1H
OCEAN"]]))
<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[ -0.01896119,  0.02223358],
       [ 0.02401174,  0.03724445],
       [-0.01896119,  0.02223358]], dtype=float32)>
```

Lưu ý rằng số hàng trong ma trận nhúng cần phải bằng kích thước từ vựng: đó là tổng số danh mục, bao gồm các danh mục đã biết cộng với các nhóm OOV (mặc định chỉ có một nhóm). Phương thức `vocabulary\_size()` của lớp `StringLookup` sẽ trả về con số này một cách thuận tiện.

MẸO

Trong ví dụ này, chúng ta đã sử dụng embedding 2D, nhưng theo nguyên tắc chung, embedding thường có từ 10 đến 300 chiều, tùy thuộc vào nhiệm vụ, kích thước từ vựng và kích thước tập dữ liệu huấn luyện. Bạn sẽ cần phải điều chỉnh siêu tham số này.

Tóm lại, giờ đây chúng ta có thể tạo ra một mô hình Keras có khả năng xử lý đặc trưng văn bản phân loại cùng với các đặc trưng số thông thường và học được một embedding cho mỗi danh mục (cũng như cho mỗi nhóm oov):

```

X_train_num, X_train_cat, y_train = [...] # tải tập dữ liệu huấn luyện X_valid_num, X_valid_cat, y_valid = [...] # và
tập
dữ liệu kiểm chứng

num_input = tf.keras.layers.Input(shape=[8], name="num") cat_input
= tf.keras.layers.Input(shape=[], dtype=tf.string, name="cat")
cat_embeddings
= lookup_and_embed(cat_input) encoded_inputs =
tf.keras.layers.concatenate([num_input, cat_embeddings]) outputs
=
tf.keras.layers.Dense(1)(encoded_inputs) model =
tf.keras.models.Model(inputs=[num_input, cat_input], outputs=[outputs])
model.compile(loss="mse",
optimizer="sgd") history = model.fit((X_train_num,
X_train_cat), y_train, epochs=5,

validation_data=((X_valid_num, X_valid_cat),
y_valid))

```

Mô hình này nhận hai đầu vào: `num_input` chứa tám đặc trưng số cho mỗi trường hợp, cộng với `cat_input` chứa một đầu vào văn bản phân loại duy nhất cho mỗi trường hợp. Mô hình sử dụng mô hình `lookup_and_embed` mà chúng ta đã tạo trước đó để mã hóa mỗi danh mục khoảng cách đến đại dương thành các embedding có thể huấn luyện tương ứng. Tiếp theo, nó nối các đầu vào số và các embedding bằng cách sử dụng hàm `concatenate()` để tạo ra các đầu vào được mã hóa hoàn chỉnh, sẵn sàng để đưa vào mạng nơ-ron. Chúng ta có thể thêm bất kỳ loại mạng nơ-ron nào tại thời điểm này, nhưng để đơn giản, chúng ta chỉ thêm một lớp đầu ra dày đặc duy nhất, và sau đó chúng ta tạo Mô hình Keras với các đầu vào và đầu ra mà chúng ta vừa định nghĩa. Tiếp theo, chúng ta biên dịch mô hình và huấn luyện nó, truyền cả đầu vào số và phân loại.

Như chúng ta đã thấy trong **Chương 10**, vì các lớp đầu vào được đặt tên là "num" và "cat", chúng ta cũng có thể truyền dữ liệu huấn luyện cho phương thức `fit()` bằng cách sử dụng một từ điển thay vì một tuple: `{"num": X_train_num, "cat": X_train_cat}`. Ngoài ra, chúng ta cũng có thể truyền một `tf.data.Dataset` chứa các batch, mỗi batch được biểu diễn dưới dạng `((X_batch_num, X_batch_cat), y_batch)` hoặc dưới dạng `({"num":`

`X_batch_num, "cat": X_batch_cat}, y_batch)`. Và tất nhiên điều tương tự cũng áp dụng cho dữ liệu xác thực.

GHI CHÚ

Mã hóa one-hot tiếp theo là lớp Dense (không có hàm kích hoạt và không có độ lệch) tương đương với lớp Embedding. Tuy nhiên, lớp Embedding sử dụng ít phép tính hơn nhiều vì nó tránh được nhiều phép nhân với số 0 – sự khác biệt về hiệu năng trở nên rõ ràng khi kích thước của ma trận nhúng tăng lên. Ma trận trọng số của lớp Dense đóng vai trò của ma trận nhúng. Ví dụ, sử dụng các vectơ one-hot có kích thước 20 và một lớp Dense với 10 đơn vị tương đương với việc sử dụng một lớp Embedding với `input_dim=20` và `output_dim=10`. Do đó, sẽ lãng phí nếu sử dụng nhiều chiều nhúng hơn số đơn vị trong lớp theo sau lớp Embedding.

Được rồi, giờ bạn đã học được cách mã hóa các đặc trưng phân loại, đã đến lúc chúng ta chuyển sự chú ý sang khâu tiền xử lý văn bản.

Xử lý trước văn bản

Keras cung cấp lớp `TextVectorization` để xử lý văn bản cơ bản. Tương tự như lớp `StringLookup`, bạn phải truyền cho nó một từ vựng khi tạo hoặc để nó tự học từ vựng từ dữ liệu huấn luyện bằng phương thức `adapt()`. Hãy xem một ví dụ:

```
>>> train_data = ["To be", "!(to be)", "That's the question", "Be, be, be."] >>>
text_vec_layer =
tf.keras.layers.TextVectorization() >>> text_vec_layer.adapt(train_data)
>>> text_vec_layer(["Be good!", "Question: be
or be?"]) <tf.Tensor: shape=(2, 4), dtype=int64, numpy= array([[2, 1, 0,
0], [6, 2, 1, 2]])>
```

Hai câu “Hãy ngoan!” và “Câu hỏi: be hay be?” được mã hóa lần lượt là `[2, 1, 0, 0]` và `[6, 2, 1, 2]`. Từ vựng được học từ 4 câu trong dữ liệu huấn luyện: “be” = 2, “to” = 3, v.v.

Để xây dựng từ vựng, phương thức `adapt()` trước tiên chuyển đổi các câu huấn luyện thành chữ thường và loại bỏ dấu câu, đó là lý do tại sao “Be”, “be,” và “be?” đều được mã hóa là “be” = 2. Tiếp theo, các câu được tách ra tại các khoảng trắng, và các từ kết quả được sắp xếp theo tần suất giảm dần, tạo ra từ vựng cuối cùng. Khi mã hóa câu, các từ không xác định được mã hóa là 1. Cuối cùng, vì câu đầu tiên ngắn hơn câu thứ hai, nên nó được thêm các số 0 vào cuối.

MẸO

Lớp `TextVectorization` có nhiều tùy chọn. Ví dụ, bạn có thể giữ nguyên chữ hoa chữ thường và dấu câu nếu muốn bằng cách đặt `standardize=None`, hoặc bạn có thể truyền bất kỳ hàm chuẩn hóa nào bạn muốn làm đối số `standardize`. Bạn có thể ngăn việc tách chuỗi bằng cách đặt `split=None`, hoặc thậm chí bạn có thể truyền hàm tách chuỗi của riêng mình. Bạn có thể đặt đối số `output_sequence_length` để đảm bảo rằng tất cả các chuỗi đầu ra đều được cắt hoặc thêm phần đệm đến độ dài mong muốn. Hoặc, bạn có thể đặt `ragged=True` để nhận được một tensor không đều thay vì một tensor thông thường. Vui lòng xem tài liệu để biết thêm các tùy chọn khác.

Các định danh từ phải được mã hóa, thông thường bằng cách sử dụng lớp `Embedding`: chúng ta sẽ thực hiện điều này trong [Chương 16](#). Ngoài ra, bạn có thể đặt đối số `output_mode` của lớp `TextVectorization` thành “multi_hot” hoặc “count” để nhận được các mã hóa tương ứng.

Tuy nhiên, việc chỉ đếm từ thường không phải là lý tưởng: thực tế, những từ như “to” hay “the” xuất hiện thường xuyên đến mức chúng hầu như không có ý nghĩa gì. Ngược lại, những từ hiếm gặp hơn như “basketball” lại mang nhiều thông tin hơn. Vì vậy, thay vì đặt `output_mode` thành “multi_hot” hoặc “count”, thường thì nên đặt nó thành “tf_idf”, viết tắt của Term-Frequency × Inverse-Document-Frequency (TF-IDF). Điều này tương tự như mã hóa đếm, nhưng những từ xuất hiện thường xuyên trong dữ liệu huấn luyện sẽ được giảm trọng số, và ngược lại, những từ hiếm gặp sẽ được tăng trọng số. Ví dụ:

```
>>> text_vec_layer =
tf.keras.layers.TextVectorization(output_mode="tf_idf") >>>
text_vec_layer.adapt(train_data)
```

```
>>> text_vec_layer(["Hãy ngoan ngoãn!", "Câu hỏi: be hay be?"])
<tf.Tensor: shape=(2, 6), dtype=float32, numpy=
mảng([[0.96725637, 0.6931472 ],          ,          ,          ,          ,
        0. [0.96725637, 1.3862944 1.0986123]], 0. 0., 0. 0.,
dtype=float32)>
```

Có nhiều biến thể TF-IDF, nhưng cách TextVectorization hoạt động như thế nào thì khác nhau.

Lớp này thực hiện điều đó bằng cách nhân số lần xuất hiện của mỗi từ với một trọng số bằng

$\log(1 + d / (f + 1))$ trong đó d là tổng số câu (hay còn gọi là,

(tài liệu) trong dữ liệu huấn luyện, và f đếm số lượng tài liệu huấn luyện này.

các câu chứa từ đã cho. Ví dụ, có $d = 4$ câu trong

dữ liệu huấn luyện, và từ “be” xuất hiện trong $f = 3$ trong số đó. Vì vậy,

Từ “be” xuất hiện hai lần trong câu “Question: be or be?”, điều này khiến câu hỏi trở nên khó hiểu.

được mã hóa thành $2 \times \log(1 + 4 / (1 + 3)) \approx 1.3862944$. Chỉ có từ “question”

Từ này xuất hiện một lần, nhưng vì nó là một từ ít phổ biến nên cách mã hóa của nó gần như tương tự.

cao: $1 \times \log(1 + 4 / (1 + 1)) \approx 1.0986123$. Lưu ý rằng trọng lượng trung bình là

Dùng để tra cứu những từ chưa biết.

Phương pháp mã hóa văn bản này rất dễ sử dụng và có thể mang lại nhiều lợi ích.

Kết quả khá tốt cho các tác vụ xử lý ngôn ngữ tự nhiên cơ bản, nhưng nó có

Một số hạn chế quan trọng: nó chỉ hoạt động với các ngôn ngữ phân tách

Đối với các từ có khoảng trắng, nó không phân biệt giữa các từ đồng âm (ví dụ: “to

so sánh giữa “bear” và “teddy bear”), điều này không hề gợi ý cho mô hình của bạn rằng những từ như vậy

“tiến hóa” và “mang tính tiến hóa” có liên quan đến nhau, v.v. Và nếu bạn sử dụng đa điểm nóng,

Nếu sử dụng mã hóa count hoặc TF-IDF, thứ tự các từ sẽ bị mất. Vậy thì chúng là gì?

Các lựa chọn khác thì sao?

Một lựa chọn khác là sử dụng **thư viện TensorFlow Text**. điều này mang lại nhiều hơn

các tính năng tiền xử lý văn bản nâng cao hơn so với TextVectorization

lớp. Ví dụ, nó bao gồm một số bộ phân tách từ con có khả năng

chia văn bản thành các mã thông báo nhỏ hơn từ, điều này giúp cho việc

mô hình để dễ dàng phát hiện hơn rằng “tiến hóa” và “mang tính tiến hóa” có

có điểm chung (chi tiết hơn về phân tách từ con trong **Chương 16**).

Một lựa chọn khác nữa là sử dụng các thành phần mô hình ngôn ngữ được huấn luyện trước. Hãy cùng xem...

Hãy nhìn điều này xem.

Sử dụng các thành phần mô hình ngôn ngữ được huấn luyện trước

Thư viện **TensorFlow Hub** TF Hub giúp bạn dễ dàng tái sử dụng các thành phần mô hình đã được huấn luyện trước trong các mô hình của riêng bạn, cho văn bản, hình ảnh, âm thanh hoặc nhiều hơn nữa. Các thành phần mô hình này được gọi là mô-đun. Chỉ cần duyệt kho lưu trữ **TF Hub**, tìm mô-đun bạn cần và sao chép ví dụ mã vào dự án của bạn, mô-đun sẽ được tự động tải xuống và đóng gói thành một lớp Keras mà bạn có thể trực tiếp đưa vào mô hình của mình. Các mô-đun thường chứa cả mã tiền xử lý và trọng số đã được huấn luyện trước, và chúng thường không yêu cầu huấn luyện thêm (nhưng tất nhiên, phần còn lại của mô hình của bạn chắc chắn sẽ cần huấn luyện).

Ví dụ, có một số mô hình ngôn ngữ được huấn luyện trước mạnh mẽ. Những mô hình mạnh nhất khá lớn (vài Gigabyte), vì vậy để làm ví dụ nhanh, chúng ta hãy sử dụng mô-đun `nnlm-en-dim50`, phiên bản 2, đây là một mô-đun khá cơ bản nhận văn bản thô làm đầu vào và xuất ra các nhúng câu 50 chiều. Hãy nhập TensorFlow Hub và sử dụng nó để tải mô-đun, sau đó sử dụng mô-đun đó để mã hóa hai câu thành vectơ:

```
>>> import tensorflow_hub as hub >>> hub_layer
= hub.KerasLayer("https://tfhub.dev/google/nnlm-en-dim50/2") >>> sentence_embeddings =
hub_layer(tf.constant(["To be", "Not to be"])) >>> sentence_embeddings.numpy().round(2)

array([[ -0.25,  0.28,  0.01,  0.1 , [...],  0.05,  0.31],
       [-0.2 ,  0.2 , -0.08,  0.02, [...], -0.04,  0.15]], dtype=float32)
```

Lớp `hub.KerasLayer` tải xuống mô-đun từ URL được cung cấp.

Mô-đun cụ thể này là một bộ mã hóa câu: nó nhận các chuỗi ký tự làm đầu vào và mã hóa mỗi chuỗi thành một vectơ duy nhất (trong trường hợp này, là một vectơ 50 chiều). Về mặt kỹ thuật, nó phân tích chuỗi ký tự (tách các từ dựa trên dấu cách) và nhúng từng từ bằng ma trận nhúng đã được huấn luyện trước trên một kho ngữ liệu khổng lồ: kho ngữ liệu Google News 7B (dài bảy tỷ từ!). Sau đó, nó tính toán giá trị trung bình của tất cả các ma trận nhúng từ, và kết quả là ma trận nhúng của câu.

Bạn chỉ cần thêm `hub_layer` này vào mô hình của mình là xong. Lưu ý rằng mô hình ngôn ngữ cụ thể này được huấn luyện trên tiếng Anh, nhưng có nhiều ngôn ngữ khác cũng như các mô hình đa ngôn ngữ.

Cuối cùng nhưng không kém phần quan trọng, thư viện Transformers mã nguồn mở tuyệt vời của [Hugging Face](#) cũng giúp dễ dàng tích hợp các thành phần mô hình ngôn ngữ mạnh mẽ vào mô hình của riêng bạn. Bạn có thể duyệt qua [HuggingFace Hub](#), chọn mô hình bạn muốn và sử dụng các ví dụ mã được cung cấp để bắt đầu. Trước đây nó chỉ chứa các mô hình ngôn ngữ, nhưng hiện nay đã mở rộng sang các mô hình hình ảnh và nhiều hơn nữa.

Chúng ta sẽ quay lại với xử lý ngôn ngữ tự nhiên một cách chi tiết hơn trong [Chương 16](#). Bây giờ, hãy cùng xem xét các lớp tiền xử lý hình ảnh của Keras.

Các lớp tiền xử lý hình ảnh

API tiền xử lý của Keras bao gồm ba lớp tiền xử lý hình ảnh:

- `tf.keras.layers.Resizing` thay đổi kích thước ảnh đầu vào thành kích thước mong muốn. Ví dụ: `Resizing(height=100, width=200)` sẽ thay đổi kích thước ảnh thành 100×200 , có thể làm biến dạng ảnh. Nếu bạn đặt `crop_to_aspect_ratio=True`, thì ảnh sẽ được cắt theo tỷ lệ khung hình mục tiêu để tránh biến dạng.
- `tf.keras.layers.Rescaling` thay đổi tỷ lệ giá trị pixel, ví dụ: `Rescaling(scale=2/255, offset=-1)` sẽ thay đổi tỷ lệ giá trị từ 0 đến 255 thành -1 đến 1.
- `tf.keras.layers.CenterCrop` cắt ảnh, chỉ giữ lại phần trung tâm có chiều cao và chiều rộng mong muốn.

Ví dụ, chúng ta hãy tải một vài hình ảnh mẫu và cắt ảnh theo tâm.

Để làm điều này, chúng ta sẽ sử dụng hàm `load_sample_images()` của Scikit-Learn, hàm này tải hai hình ảnh màu: một hình ảnh về một ngôi chùa Trung Quốc và một hình ảnh khác về...

flower (thao tác này yêu cầu thư viện Pillow, thư viện này thường đã được cài đặt nếu bạn đang sử dụng Colab hoặc nếu bạn đã làm theo hướng dẫn cài đặt).

```
from sklearn.datasets import load_sample_images

images = load_sample_images()["images"] crop_image_layer
= tf.keras.layers.CenterCrop(height=100, width=100) cropped_images =

crop_image_layer(images)
```

Keras cũng bao gồm một số lớp để tăng cường dữ liệu, chẳng hạn như RandomCrop, RandomFlip, RandomTranslation, RandomRotation, RandomZoom, RandomHeight, RandomWidth và RandomContrast. Các lớp này chỉ hoạt động trong quá trình huấn luyện và chúng áp dụng ngẫu nhiên một số phép biến đổi cho ảnh đầu vào (tên của chúng đã nói lên tất cả). Tăng cường dữ liệu sẽ làm tăng kích thước tập dữ liệu huấn luyện một cách nhân tạo, điều này thường dẫn đến hiệu suất được cải thiện, miễn là các ảnh đã được biến đổi trông giống như ảnh thật (chưa được tăng cường). Chúng ta sẽ tìm hiểu kỹ hơn về xử lý ảnh trong chương tiếp theo.

GHI CHÚ

Về cơ bản, các lớp tiền xử lý của Keras dựa trên API cấp thấp của TensorFlow. Ví dụ, lớp Chuẩn hóa sử dụng `tf.nn.moments()` để tính cả giá trị trung bình và phương sai, lớp Rời rạc hóa sử dụng `tf.raw_ops.Bucketize()`, Mã hóa phân loại sử dụng `tf.math.bincount()`, Tra cứu số nguyên và Tra cứu chuỗi sử dụng gói `tf.lookup`, Băm và Vector hóa văn bản sử dụng một số toán tử từ gói `tf.strings`, Nhúng sử dụng `tf.nn.embedding_lookup()`, và các lớp tiền xử lý hình ảnh sử dụng các toán tử từ gói `tf.image`. Nếu API tiền xử lý của Keras không đáp ứng đủ nhu cầu của bạn, đôi khi bạn có thể cần sử dụng trực tiếp API cấp thấp của TensorFlow.

Bây giờ chúng ta hãy xem xét một cách khác để tải dữ liệu dễ dàng và hiệu quả trong TensorFlow.

Dự án Bộ dữ liệu TensorFlow (TFDS)

Bộ dữ liệu TensorFlow Dự án này giúp việc tải các tập dữ liệu phổ biến trở nên rất dễ dàng, từ những tập dữ liệu nhỏ như MNIST hoặc Fashion MNIST đến những tập dữ liệu khổng lồ như ImageNet (bạn sẽ cần khá nhiều dung lượng ổ đĩa!). Danh sách bao gồm các tập dữ liệu hình ảnh, tập dữ liệu văn bản (bao gồm cả tập dữ liệu dịch thuật), tập dữ liệu âm thanh và video, chuỗi thời gian, và nhiều hơn nữa. Bạn có thể truy cập <https://hom1.info/tfds> Để xem danh sách đầy đủ, cùng với mô tả của từng tập dữ liệu, bạn cũng có thể tham khảo thêm phần " [Hiểu rõ dữ liệu của bạn](#)". Đây là một công cụ để khám phá và hiểu nhiều bộ dữ liệu do TFDS cung cấp.

TFDS không được tích hợp sẵn trong TensorFlow, nhưng nếu bạn đang sử dụng Colab hoặc đã làm theo hướng dẫn cài đặt tại <https://hom1.info/install>, Vậy là nó đã được cài đặt rồi. Sau đó, bạn có thể nhập `tensorflow_datasets`, thường là dưới dạng `tfds`, rồi gọi hàm `tfds.load()`, hàm này sẽ tải xuống dữ liệu bạn muốn (trừ khi dữ liệu đã được tải xuống trước đó) và trả về dữ liệu dưới dạng một từ điển các tập dữ liệu (thường là một tập dữ liệu để huấn luyện và một tập dữ liệu để kiểm tra, nhưng điều này phụ thuộc vào tập dữ liệu bạn chọn). Ví dụ, hãy tải xuống MNIST:

```
import tensorflow_datasets as tfds

datasets = tfds.load(name="mnist")
mnist_train, mnist_test = datasets["train"], datasets["test"]
```

Sau đó, bạn có thể áp dụng bất kỳ phép biến đổi nào bạn muốn (thường là xáo trộn, gom nhóm và tải trước), và bạn đã sẵn sàng huấn luyện mô hình của mình. Dưới đây là một ví dụ đơn giản:

```
for batch in mnist_train.shuffle(10_000,
seed=42).batch(32).prefetch(1):
    images = batch["image"]
    labels = batch["label"] #
    [...] làm gì đó với hình ảnh và nhãn
```

MẸO

Hàm `load()` có thể xáo trộn các tập tin mà nó tải xuống: chỉ cần đặt `shuffle_files=True`. Tuy nhiên, điều này có thể chưa đủ, vì vậy tốt nhất là nên xáo trộn dữ liệu huấn luyện thêm một chút nữa.

Lưu ý rằng mỗi mục trong tập dữ liệu là một từ điển chứa cả các đặc trưng và nhãn. Nhưng Keras mong đợi mỗi mục là một bộ dữ liệu gồm hai phần tử (một lần nữa, các đặc trưng và nhãn). Bạn có thể chuyển đổi tập dữ liệu bằng phương thức `map()`, như sau:

```
mnist_train = mnist_train.shuffle(buffer_size=10_000,
seed=42).batch(32)
mnist_train = mnist_train.map(lambda items: (items["image"],
items["label"]))
mnist_train = mnist_train.prefetch(1)
```

Nhưng sẽ đơn giản hơn nếu bạn yêu cầu hàm `load()` thực hiện việc này bằng cách đặt `as_supervised=True` (rõ ràng điều này chỉ hoạt động với các tập dữ liệu đã được gắn nhãn).

Cuối cùng, TFDS cung cấp một cách thuận tiện để chia dữ liệu bằng cách sử dụng đối số `'split'`. Ví dụ, nếu bạn muốn sử dụng 90% tập dữ liệu huấn luyện để huấn luyện, 10% còn lại để xác thực và toàn bộ tập dữ liệu kiểm tra để kiểm thử, thì bạn có thể đặt `'split=["train[:90%]", "train[90%:]", "test"]`. Hàm `'load()'` sẽ trả về cả ba tập dữ liệu. Sau đây là một ví dụ hoàn chỉnh, tải và chia tập dữ liệu MNIST bằng TFDS, sau đó sử dụng các tập dữ liệu này để huấn luyện và đánh giá một mô hình Keras đơn giản:

```
train_set, valid_set, test_set =

    tfds.load( name="mnist", split=["train[:90%]",
"train[90%:]", "test"], as_supervised=True

) train_set = train_set.shuffle(buffer_size=10_000,
seed=42).batch(32).prefetch(1)
valid_set = valid_set.batch(32).cache()
test_set = test_set.batch(32).cache()
```

```

tf.random.set_seed(42)
model =

    tf.keras.Sequential([ tf.keras.layers.Flatten(input_shape=(28, 28)), tf.keras.l

]) model.compile(loss="sparse_categorical_crossentropy",
optimizer="nadam",
                metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set,
epochs=5)
test_loss, test_accuracy = model.evaluate(test_set)

```

Chúc mừng, bạn đã hoàn thành chương khá chuyên sâu này! Có thể bạn cảm thấy nó hơi xa rời vẻ đẹp trừu tượng của mạng nơ-ron, nhưng thực tế là Học sâu thường liên quan đến lượng dữ liệu lớn, và việc biết cách tải, phân tích và tiền xử lý dữ liệu một cách hiệu quả là một kỹ năng quan trọng cần có. Trong chương tiếp theo, chúng ta sẽ xem xét mạng nơ-ron tích chập, một trong những kiến trúc mạng nơ-ron thành công nhất cho xử lý ảnh và nhiều ứng dụng khác.

Bài tập

1. Tại sao bạn lại muốn sử dụng API `tf.data`?
2. Việc chia một tập dữ liệu lớn thành nhiều tập tin mang lại những lợi ích gì?
3. Trong quá trình huấn luyện, làm thế nào bạn có thể biết được đường dẫn đầu vào của mình là...
Điểm nghẽn? Bạn có thể làm gì để khắc phục nó?
4. Bạn có thể lưu bất kỳ dữ liệu nhị phân nào vào tệp `TFRecord` hay chỉ dữ liệu được tuần tự hóa?
Protocol buffers?
5. Tại sao bạn lại phải mất công chuyển đổi tất cả dữ liệu của mình sang định dạng `protobuf` ví dụ? Tại sao không sử dụng định nghĩa `protobuf` của riêng bạn?
6. Khi sử dụng `TFRecords`, bạn muốn kích hoạt tính năng nén khi nào? Tại sao không thực hiện việc này một cách có hệ thống?

7. Dữ liệu có thể được xử lý sơ bộ trực tiếp khi ghi các tệp dữ liệu, hoặc trong quy trình `tf.data`, hoặc trong các lớp xử lý sơ bộ bên trong mô hình của bạn. Bạn có thể liệt kê một vài ưu điểm và nhược điểm của mỗi tùy chọn không?
8. Hãy nêu một vài cách phổ biến để mã hóa các đặc điểm văn bản phân loại.
Còn về phần văn bản thì sao?
9. Tải bộ dữ liệu Fashion MNIST (đã giới thiệu trong [Chương 10](#)); chia nó thành bộ huấn luyện, bộ xác thực và bộ kiểm tra; xáo trộn bộ huấn luyện; và lưu mỗi bộ dữ liệu vào nhiều tệp `TFRecord`. Mỗi bản ghi phải là một protobuf Example được tuần tự hóa với hai đặc trưng: hình ảnh được tuần tự hóa (sử dụng `tf.io.serialize_tensor()` để tuần tự hóa từng hình ảnh) và nhãn. Sau đó, sử dụng `tf.data` để tạo một bộ dữ liệu hiệu quả cho mỗi bộ. Cuối cùng, sử dụng mô hình Keras để huấn luyện các bộ dữ liệu này, bao gồm một lớp tiền xử lý để chuẩn hóa từng đặc trưng đầu vào. Cố gắng làm cho quy trình đầu vào hiệu quả nhất có thể, sử dụng TensorBoard để trực quan hóa dữ liệu phân tích.
10. Trong bài tập này, bạn sẽ tải xuống một tập dữ liệu, chia nhỏ nó và tạo ra một...
Sử dụng `tf.data.Dataset` để tải và tiền xử lý dữ liệu một cách hiệu quả, sau đó xây dựng và huấn luyện mô hình phân loại nhị phân có chứa lớp Embedding:
 - a. Tải xuống **Bộ dữ liệu đánh giá phim lớn**, trong đó có 50.000 bài đánh giá phim từ **Internet Movie Database**. Dữ liệu được tổ chức trong hai thư mục, train và test, mỗi thư mục chứa một thư mục con pos với 12.500 đánh giá tích cực và một thư mục con neg với 12.500 đánh giá tiêu cực. Mỗi đánh giá được lưu trữ trong một tệp văn bản riêng biệt. Có các tệp và thư mục khác (bao gồm cả mô hình bag-of-words đã được xử lý trước), nhưng chúng ta sẽ bỏ qua chúng trong bài tập này.
 - b. Chia tập dữ liệu thử nghiệm thành tập dữ liệu xác thực (15.000) và tập dữ liệu thử nghiệm (10.000).
 - c. Sử dụng `tf.data` để tạo một tập dữ liệu hiệu quả cho mỗi bộ dữ liệu.

- d. Tạo mô hình phân loại nhị phân, sử dụng lớp `TextVectorization` để xử lý trước từng bài đánh giá.
- e. Thêm một lớp `Embedding` và tính toán giá trị trung bình của embedding cho mỗi bài đánh giá, nhân với căn bậc hai của số từ (xem [Chương 16](#)). Giá trị trung bình đã được điều chỉnh này sau đó có thể được truyền đến phần còn lại của mô hình.
- f. Huấn luyện mô hình và xem độ chính xác đạt được. Cố gắng tối ưu hóa. Tối ưu hóa quy trình của bạn để quá trình đào tạo diễn ra nhanh nhất có thể.
- g. Sử dụng TFDS để tải cùng một tập dữ liệu dễ dàng hơn: `tfds.load("imdb_reviews")`.

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://homl.info/colab3>.

-
1. Hãy tưởng tượng một bộ bài đã được sắp xếp ở bên trái bạn: giả sử bạn lấy ba lá bài trên cùng và xáo trộn chúng, sau đó chọn ngẫu nhiên một lá và đặt nó sang bên phải, giữ hai lá còn lại trong tay. Lấy thêm một lá bài nữa ở bên trái, xáo trộn ba lá bài trong tay và chọn ngẫu nhiên một lá, rồi đặt nó sang bên phải. Khi bạn làm xong tất cả các lá bài như vậy, bạn sẽ có một bộ bài ở bên phải: bạn nghĩ nó có được xáo trộn hoàn hảo không?
 2. Nhìn chung, chỉ cần tải trước một lô dữ liệu là đủ, nhưng trong một số trường hợp, bạn có thể cần tải trước nhiều lô hơn. Ngoài ra, bạn có thể để TensorFlow tự động quyết định bằng cách truyền tham số `tf.data.AUTOTUNE` cho hàm `prefetch()`.
 3. Nhưng hãy xem thử hàm thử nghiệm `tf.data.experimental.prefetch_to_device()`, hàm này có thể tải trước dữ liệu trực tiếp vào GPU. Bất kỳ hàm hoặc lớp TensorFlow nào có từ "experimental" trong tên đều có thể thay đổi mà không cần báo trước trong các phiên bản tương lai. Nếu một hàm thử nghiệm bị lỗi, hãy thử xóa từ "experimental": có thể nó đã được chuyển sang API cốt lõi. Nếu không, vui lòng kiểm tra notebook, vì tôi sẽ đảm bảo nó chứa mã cập nhật nhất.
 4. Vì các đối tượng protobuf được thiết kế để tuần tự hóa và truyền tải, chúng được gọi là các thông điệp.
 5. Chương này chứa những kiến thức tối thiểu bạn cần biết về protobuf để sử dụng `TfRecords`. Để tìm hiểu thêm về protobufs, vui lòng truy cập <https://homl.info/protobuf>.
 6. Tomas Mikolov và cộng sự, "Biểu diễn phân tán của từ và cụm từ và của chúng" "Tính cấu thành," Kỷ yếu Hội nghị Quốc tế lần thứ 26 về Hệ thống Xử lý Thông tin Thần kinh 2 (2013): 3111-3119.

7. Malvina Nissim và cộng sự, "Công bằng hơn giặt gân: Đàn ông với bác sĩ cũng như phụ nữ với bác sĩ" Bác sĩ," bản in trước arXiv arXiv:1905.09866 (2019).
8. TensorFlow Hub không được tích hợp sẵn trong TensorFlow, nhưng nếu bạn đang sử dụng Colab hoặc đã làm theo hướng dẫn cài đặt tại <https://hom1.info/install>, Vậy là nó đã được cài đặt rồi.
9. Nói một cách chính xác, phép nhúng câu bằng giá trị trung bình của phép nhúng từ nhân với căn bậc hai của số từ trong câu. Điều này bù đắp cho thực tế là giá trị trung bình của n vectơ ngẫu nhiên sẽ ngắn hơn khi n tăng lên.
10. Đối với ảnh có dung lượng lớn, bạn có thể sử dụng `tf.io.encode_jpeg()` thay thế. Cách này sẽ tiết kiệm được nhiều dung lượng, nhưng chất lượng ảnh sẽ giảm đi một chút.

Chương 14. Thị giác máy tính sâu sử dụng mạng nơ-ron tích chập

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất-nội dung thô và chưa chỉnh sửa của tác giả khi họ viết-nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 14 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Mặc dù siêu máy tính Deep Blue của IBM đã đánh bại nhà vô địch cờ vua thế giới Garry Kasparov vào năm 1996, nhưng mãi đến gần đây, máy tính mới có thể thực hiện một cách đáng tin cậy những nhiệm vụ tưởng chừng như tầm thường như phát hiện một chú chó con trong ảnh hoặc nhận dạng lời nói. Tại sao những nhiệm vụ này lại dễ dàng đối với con người chúng ta? Câu trả lời nằm ở chỗ nhận thức phần lớn diễn ra bên ngoài phạm vi ý thức của chúng ta, bên trong các mô-đun thị giác, thính giác và các giác quan chuyên biệt khác trong não bộ. Khi thông tin cảm giác đến được ý thức của chúng ta, nó đã được tô điểm bằng các đặc điểm cấp cao; ví dụ, khi bạn nhìn vào một bức ảnh của một chú chó con dễ thương, bạn không thể chọn không nhìn thấy chú chó con đó, không nhận thấy sự dễ thương của nó. Bạn cũng không thể giải thích làm thế nào bạn nhận ra một chú chó con dễ thương; điều đó đơn giản là hiển nhiên đối với bạn. Do đó, chúng ta không thể tin tưởng vào kinh nghiệm chủ quan của mình: nhận thức không hề tầm thường chút nào, và để hiểu nó, chúng ta phải xem xét cách thức hoạt động của các mô-đun cảm giác của chúng ta.

Mạng nơ-ron tích chập (CNN) ra đời từ nghiên cứu về vỏ não thị giác và đã được sử dụng trong nhận dạng hình ảnh bằng máy tính từ những năm 1980. Trong mười năm qua, nhờ sự gia tăng sức mạnh tính toán, lượng dữ liệu huấn luyện sẵn có và các thủ thuật được trình bày trong [Chương 11](#) để huấn luyện mạng sâu, CNN đã đạt được hiệu suất vượt trội so với con người trong một số tác vụ thị giác phức tạp. Chúng cung cấp sức mạnh cho các dịch vụ tìm kiếm hình ảnh, xe tự lái, hệ thống phân loại video tự động, và nhiều hơn nữa. Hơn nữa, CNN không chỉ giới hạn ở nhận thức thị giác: chúng cũng thành công trong nhiều nhiệm vụ khác, chẳng hạn như nhận dạng giọng nói và xử lý ngôn ngữ tự nhiên. Tuy nhiên, hiện tại chúng ta sẽ tập trung vào các ứng dụng thị giác.

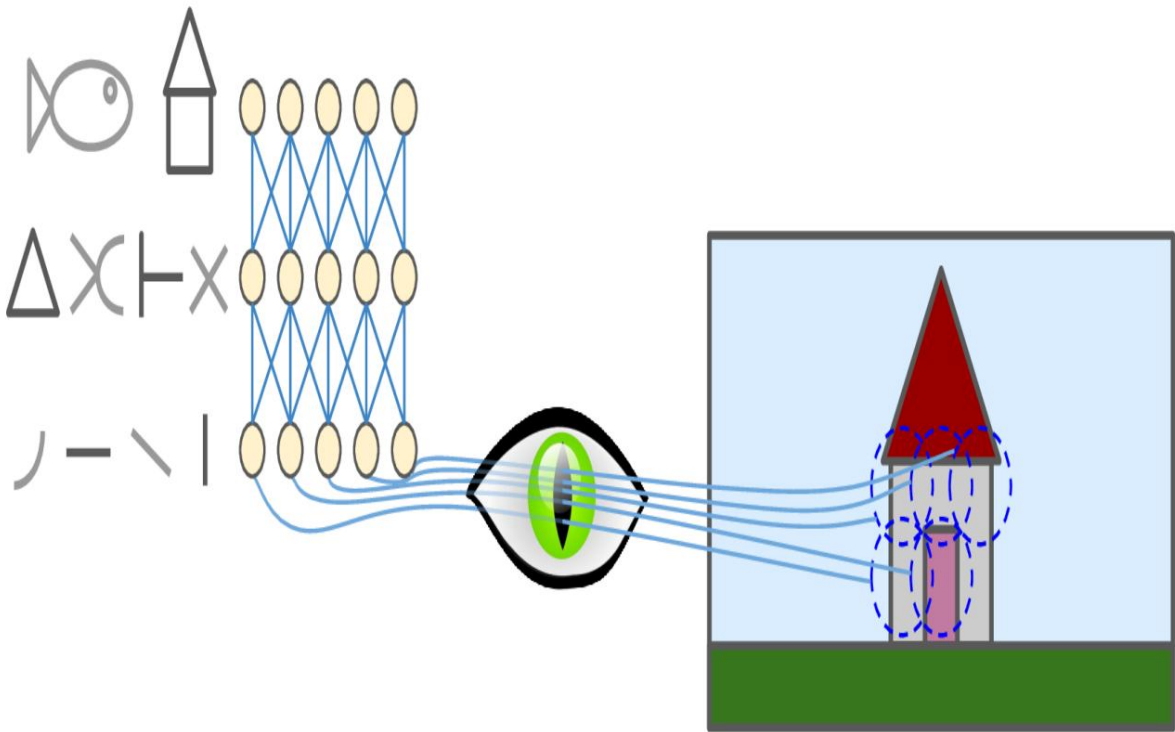
Trong chương này, chúng ta sẽ tìm hiểu nguồn gốc của mạng nơ-ron tích chập (CNN), các khối cấu tạo của chúng trông như thế nào và cách triển khai chúng bằng Keras. Sau đó, chúng ta sẽ thảo luận về một số kiến trúc CNN tốt nhất, cũng như các tác vụ hình ảnh khác, bao gồm phát hiện đối tượng (phân loại nhiều đối tượng trong một hình ảnh và đặt các khung bao quanh chúng) và phân đoạn ngữ nghĩa (phân loại mỗi pixel theo lớp của đối tượng mà nó thuộc về).

Cấu trúc của vỏ não thị giác

David H. Hubel và Torsten Wiesel đã thực hiện một loạt thí nghiệm trên mèo vào năm 1958 và 1959 (và vài năm sau đó trên khỉ), mang lại những hiểu biết quan trọng về cấu trúc của vỏ não thị giác (các tác giả đã nhận được giải Nobel Sinh lý học hoặc Y học năm 1981 cho công trình của họ). Cụ thể, họ đã chỉ ra rằng nhiều tế bào thần kinh trong vỏ não thị giác có trường tiếp nhận cục bộ nhỏ, nghĩa là chúng chỉ phản ứng với các kích thích thị giác nằm trong một vùng giới hạn của trường thị giác (xem [Hình 14-1](#), trong đó trường tiếp nhận cục bộ của năm tế bào thần kinh được biểu thị bằng các vòng tròn nét đứt).

Vùng tiếp nhận của các tế bào thần kinh khác nhau có thể chồng chéo lên nhau, và cùng nhau chúng tạo nên toàn bộ trường thị giác.

Hơn nữa, các tác giả đã chỉ ra rằng một số tế bào thần kinh chỉ phản ứng với hình ảnh các đường thẳng nằm ngang, trong khi những tế bào khác chỉ phản ứng với các đường thẳng có hướng khác nhau (hai tế bào thần kinh có thể có cùng trường tiếp nhận nhưng phản ứng với các hướng đường thẳng khác nhau). Họ cũng nhận thấy rằng một số tế bào thần kinh có trường tiếp nhận lớn hơn và chúng phản ứng với các mẫu phức tạp hơn là sự kết hợp của các mẫu cấp thấp hơn. Những quan sát này đã dẫn đến ý tưởng rằng các tế bào thần kinh cấp cao hơn dựa trên đầu ra của các tế bào thần kinh cấp thấp hơn lân cận (trong [Hình 14-1](#), hãy lưu ý rằng mỗi tế bào thần kinh chỉ được kết nối với các tế bào thần kinh gần đó từ lớp trước). Kiến trúc mạnh mẽ này có khả năng phát hiện tất cả các loại mẫu phức tạp trong bất kỳ khu vực nào của trường thị giác.



Hình 14-1. Các tế bào thần kinh sinh học trong vỏ thị giác phản ứng với các mô hình cụ thể trong các vùng nhỏ của trường thị giác được gọi là trường tiếp nhận; khi tín hiệu thị giác truyền qua các mô-đun não liên tiếp, các tế bào thần kinh sẽ phản ứng với các mô hình phức tạp hơn trong các trường tiếp nhận lớn hơn.

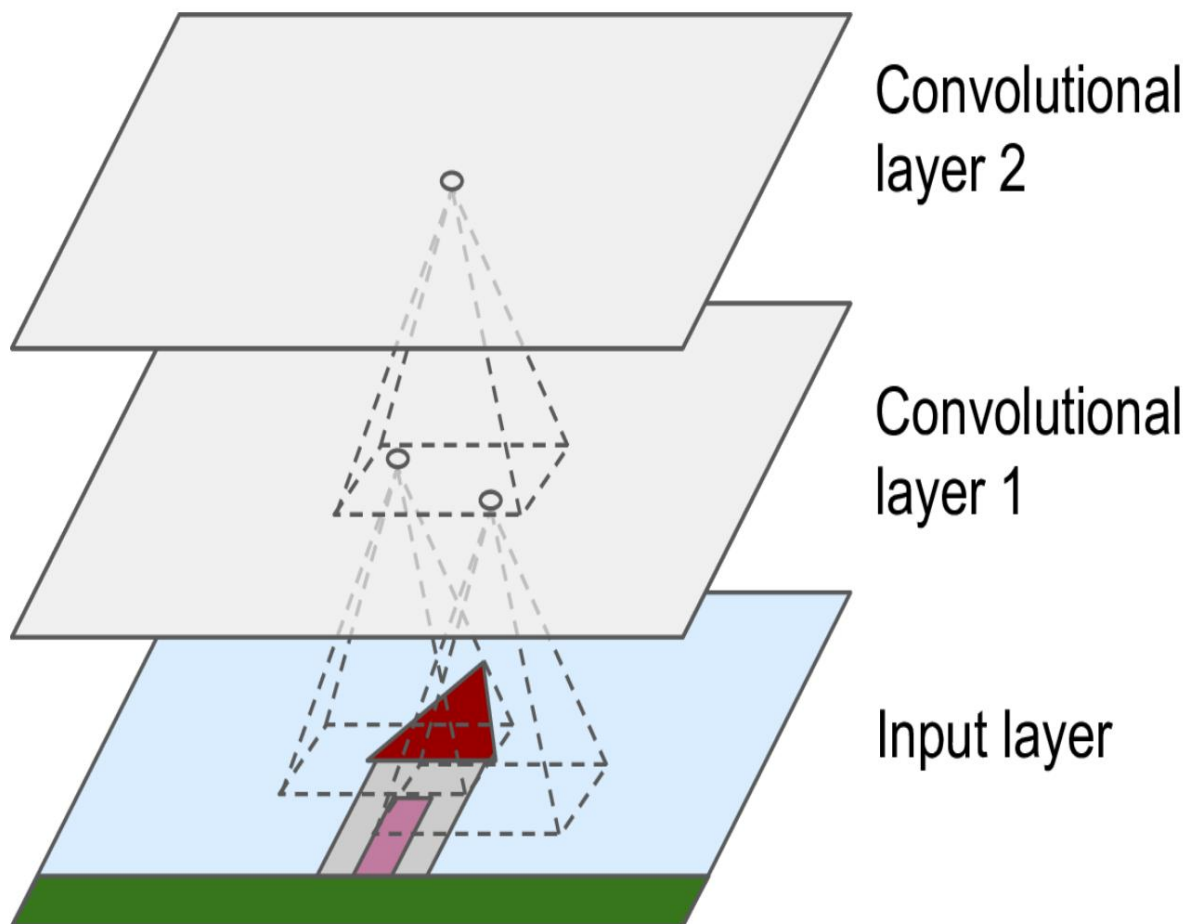
Những nghiên cứu về vỏ thị giác đã truyền cảm hứng cho **neocognitron**⁴. Được giới thiệu vào năm 1980, nó dần dần phát triển thành cái mà ngày nay chúng ta gọi là mạng nơ-ron tích chập. Một cột mốc quan trọng là **bài báo năm 1998**⁵. Do Yann LeCun và cộng sự phát triển, kiến trúc LeNet-5 nổi tiếng đã được sử dụng rộng rãi trong ngành ngân hàng để nhận dạng chữ số viết tay trên séc. Kiến trúc này có một số khối xây dựng quen thuộc, chẳng hạn như các lớp kết nối đầy đủ và hàm kích hoạt sigmoid, nhưng nó cũng giới thiệu hai khối xây dựng mới: các lớp tích chập và các lớp gộp. Chúng ta hãy cùng tìm hiểu về chúng ngay bây giờ.

GHI CHÚ

Tại sao không đơn giản sử dụng mạng nơ-ron sâu với các lớp kết nối đầy đủ cho các tác vụ nhận dạng hình ảnh? Thật không may, mặc dù phương pháp này hoạt động tốt với các hình ảnh nhỏ (ví dụ: MNIST), nhưng nó lại gặp vấn đề với các hình ảnh lớn hơn do số lượng tham số khổng lồ mà nó yêu cầu. Ví dụ, một hình ảnh 100 × 100 pixel có 10.000 pixel, và nếu lớp đầu tiên chỉ có 1.000 nơ-ron (điều này đã hạn chế nghiêm trọng lượng thông tin được truyền đến lớp tiếp theo), thì tổng cộng sẽ có 10 triệu kết nối. Và đó chỉ là lớp đầu tiên. Mạng nơ-ron tích chập (CNN) giải quyết vấn đề này bằng cách sử dụng các lớp kết nối một phần và chia sẻ trọng số.

Các lớp tích chập: Khối cấu tạo quan

trọng nhất của mạng CNN là lớp tích chập: các nơ-ron trong lớp tích chập đầu tiên không được kết nối với mọi pixel trong ảnh đầu vào (như trong các lớp đã thảo luận ở các chương trước), mà chỉ được kết nối với các pixel trong vùng tiếp nhận của chúng (xem **Hình 14-2**)⁶. Ngược lại, mỗi nơ-ron trong lớp tích chập thứ hai chỉ được kết nối với các nơ-ron nằm trong một hình chữ nhật nhỏ ở lớp đầu tiên. Kiến trúc này cho phép mạng tập trung vào các đặc trưng nhỏ, cấp thấp trong lớp ẩn đầu tiên, sau đó kết hợp chúng thành các đặc trưng lớn hơn, cấp cao hơn trong lớp ẩn tiếp theo, và cứ thế tiếp tục. Cấu trúc phân cấp này phổ biến trong các hình ảnh thực tế, đó là một trong những lý do tại sao mạng CNN hoạt động rất tốt trong nhận dạng hình ảnh.

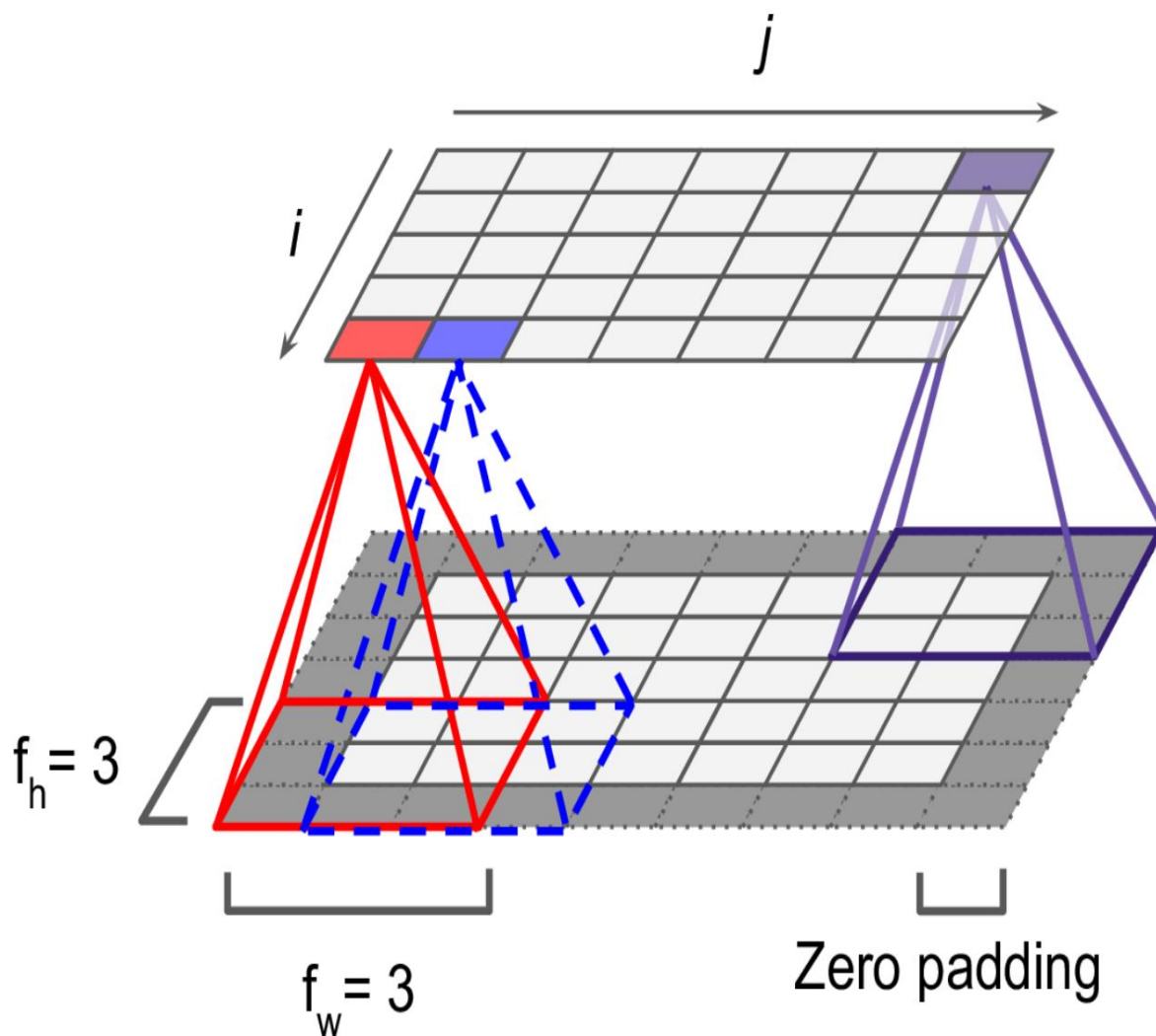


Hình 14-2. Các lớp CNN với trường tiếp nhận cục bộ hình chữ nhật.

GHI CHÚ

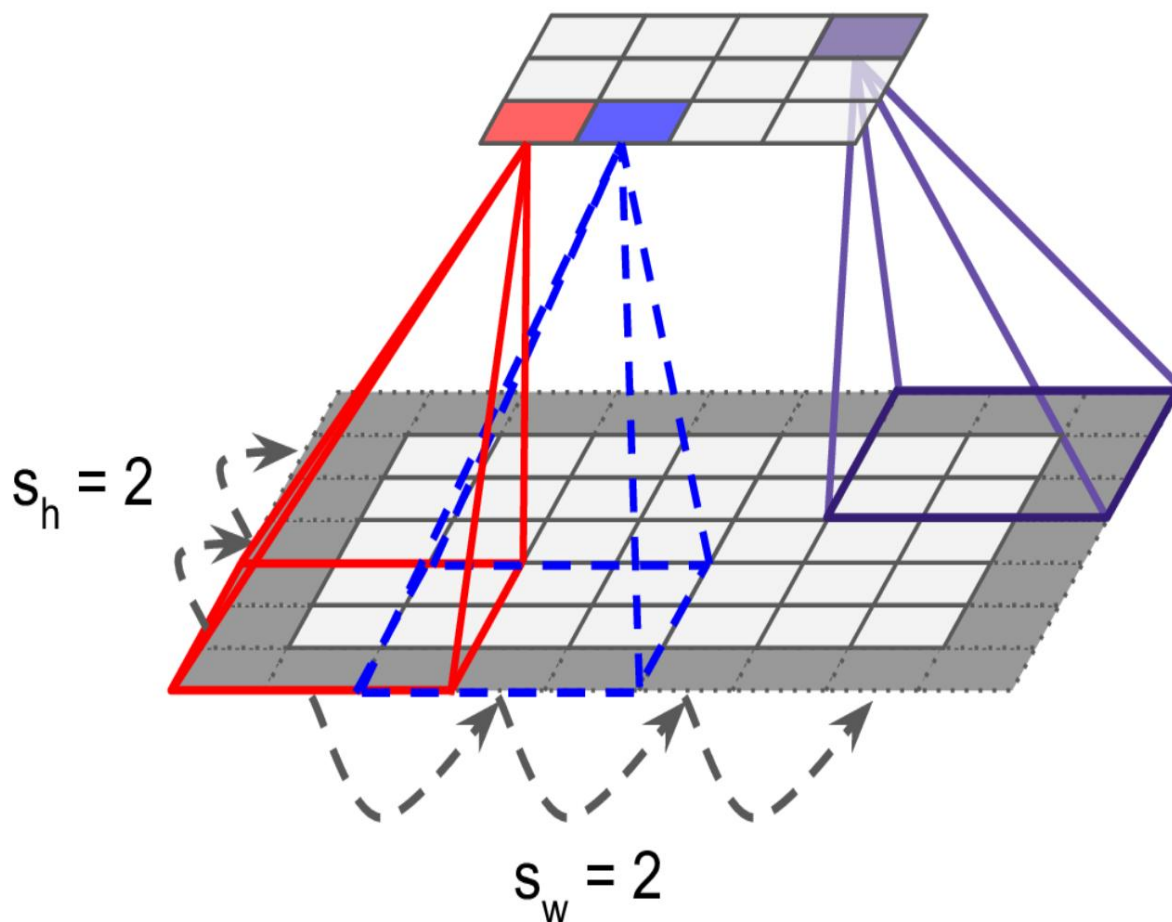
Tất cả các mạng nơ-ron đa lớp mà chúng ta đã xem xét cho đến nay đều có các lớp được cấu tạo từ một hàng dài các nơ-ron, và chúng ta phải làm phẳng ảnh đầu vào. chuyển đổi chúng thành dạng 1D trước khi đưa vào mạng nơ-ron. Trong mạng nơ-ron tích chập (CNN), mỗi lớp được biểu diễn dưới dạng 2D, giúp việc ghép nối các nơ-ron dễ dàng hơn. các đầu vào tương ứng của chúng.

Một nơ-ron nằm ở hàng i , cột j của một lớp nhất định được kết nối với đầu ra của các nơ-ron trong lớp trước đó. nằm ở các hàng từ $i - f + 1$ đến $i + f - 1$, các cột từ $j - f + 1$ đến $j + f - 1$, trong đó f là chiều cao và chiều rộng của trường tiếp nhận. (xem Hình 14-3). Để một lớp có cùng chiều cao và chiều rộng với lớp trước đó, người ta thường thêm vào... Các số 0 được thêm vào xung quanh các giá trị đầu vào, như thể hiện trong sơ đồ. Điều này được gọi là thêm số 0 đệm.



Hình 14-3. Các kết nối giữa các lớp và phân đệm bằng không.

Cũng có thể kết nối một lớp đầu vào lớn với một lớp nhỏ hơn nhiều bằng cách giãn cách các trường tiếp nhận, như như thể hiện trong Hình 14-4. Điều này làm giảm đáng kể độ phức tạp tính toán của mô hình. Theo chiều ngang hoặc chiều dọc Bước nhảy từ vùng tiếp nhận này sang vùng tiếp nhận tiếp theo được gọi là stride. Trong sơ đồ, lớp đầu vào 5×7 (cộng thêm 0) (phân đệm) được kết nối với lớp 3×4 , sử dụng các trường tiếp nhận 3×3 và bước nhảy là 2 (trong ví dụ này, bước nhảy là...) (Giống nhau ở cả hai chiều, nhưng không nhất thiết phải vậy). Một nơ-ron nằm ở hàng i , cột j trong lớp trên là được kết nối với đầu ra của các nơ-ron ở lớp trước nằm trong các hàng $i \times s$ đến $i \times s + f - 1$, cột $j \times s$ đến $j \times s + f - 1$, trong đó s là s_w và s_h là s_h bước nhảy dọc và ngang.

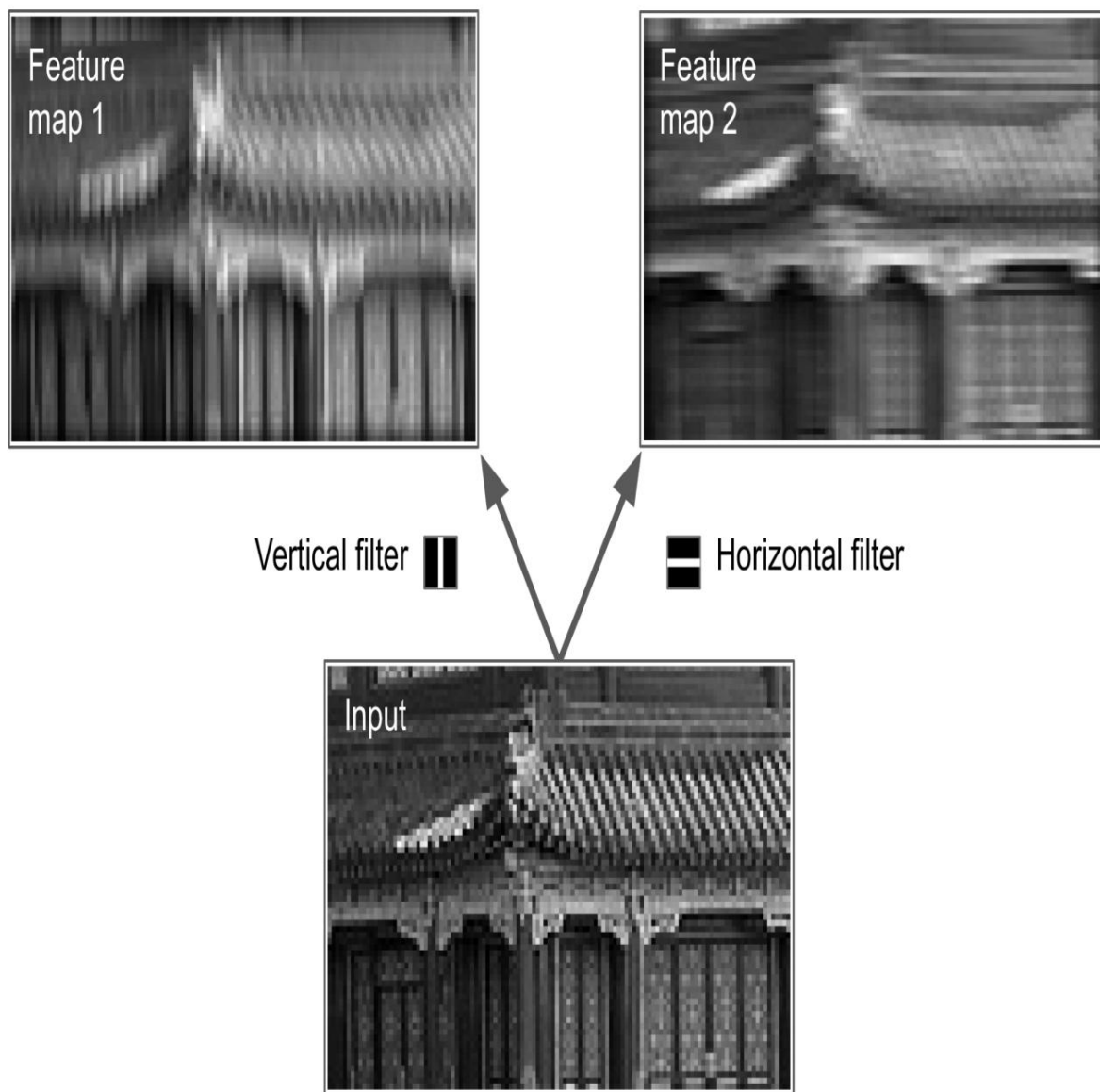


Hình 14-4. Giảm chiều bằng cách sử dụng bước nhảy 2.

Bộ lọc

Trọng số của một nơ-ron có thể được biểu diễn dưới dạng một hình ảnh nhỏ có kích thước bằng trường tiếp nhận. Ví dụ, [Hình 14-5](#) cho thấy hai tập hợp trọng số khả thi, được gọi là bộ lọc (hoặc nhân tích chập, hoặc chỉ là nhân). Bộ lọc đầu tiên được biểu diễn dưới dạng một hình vuông màu đen với một đường thẳng đứng màu trắng ở giữa (đó là một ma trận 7×7 toàn số 0 ngoại trừ cột trung tâm, toàn số 1); các nơ-ron sử dụng các trọng số này sẽ bỏ qua mọi thứ trong trường tiếp nhận của chúng ngoại trừ đường thẳng đứng ở giữa (vì tất cả các đầu vào sẽ được nhân với 0, ngoại trừ những đầu vào nằm trên đường thẳng đứng ở giữa). Bộ lọc thứ hai là một hình vuông màu đen với một đường thẳng nằm ngang màu trắng ở giữa. Các nơ-ron sử dụng các trọng số này sẽ bỏ qua mọi thứ trong trường tiếp nhận của chúng ngoại trừ đường thẳng nằm ngang ở giữa.

Nếu tất cả các nơ-ron trong một lớp sử dụng cùng một bộ lọc đường thẳng đứng (và cùng một hệ số thiên vị), và bạn đưa vào mạng hình ảnh đầu vào như trong [Hình 14-5](#) (hình ảnh phía dưới), lớp đó sẽ xuất ra hình ảnh phía trên bên trái. Lưu ý rằng các đường thẳng đứng màu trắng được làm nổi bật trong khi phần còn lại bị làm mờ. Tương tự, hình ảnh phía trên bên phải là kết quả bạn nhận được nếu tất cả các nơ-ron sử dụng cùng một bộ lọc đường thẳng ngang; lưu ý rằng các đường thẳng ngang màu trắng được làm nổi bật trong khi phần còn lại bị làm mờ. Như vậy, một lớp chứa đầy các nơ-ron sử dụng cùng một bộ lọc sẽ xuất ra một bản đồ đặc trưng, làm nổi bật các vùng trong hình ảnh kích hoạt bộ lọc nhiều nhất. Nhưng đừng lo lắng, bạn sẽ không phải tự định nghĩa các bộ lọc: thay vào đó, trong quá trình huấn luyện, lớp tích chập sẽ tự động học các bộ lọc hữu ích nhất cho nhiệm vụ của nó, và các lớp phía trên sẽ học cách kết hợp chúng thành các mẫu phức tạp hơn.



Hình 14-5. Áp dụng hai bộ lọc khác nhau để thu được hai bản đồ đặc trưng.

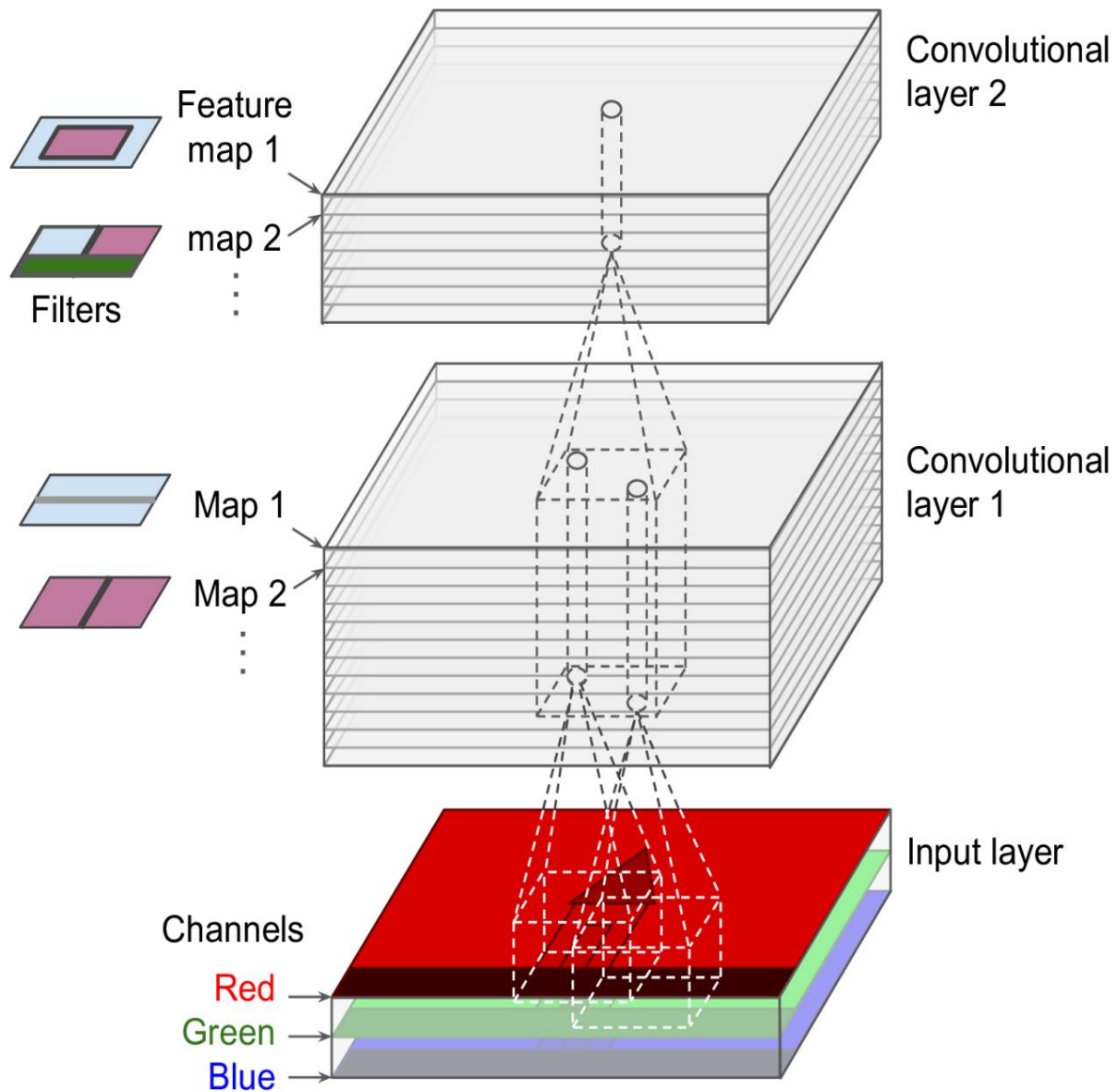
Ghép chồng nhiều bản đồ đặc trưng

Cho đến nay, để đơn giản, tôi đã biểu diễn đầu ra của mỗi lớp tích chập dưới dạng một lớp 2D, nhưng trên thực tế, một lớp tích chập có nhiều bộ lọc (bạn quyết định số lượng) và xuất ra một bản đồ đặc trưng cho mỗi bộ lọc, vì vậy nó được biểu diễn chính xác hơn trong không gian 3D (xem [Hình 14-6](#)). Nó có một nơ-ron cho mỗi pixel trong mỗi bản đồ đặc trưng, và tất cả các nơ-ron trong một bản đồ đặc trưng nhất định đều chia sẻ cùng các tham số (tức là cùng một kernel và hệ số bias). Các nơ-ron trong các bản đồ đặc trưng khác nhau sử dụng các tham số khác nhau. Trường tiếp nhận của một nơ-ron giống như đã mô tả trước đó, nhưng nó mở rộng trên tất cả các bản đồ đặc trưng của lớp trước đó. Tóm lại, một lớp tích chập đồng thời áp dụng nhiều bộ lọc có thể huấn luyện được cho đầu vào của nó, giúp nó có khả năng phát hiện nhiều đặc trưng ở bất kỳ đâu trong đầu vào của nó.

GHI CHÚ

Việc tất cả các nơ-ron trong bản đồ đặc trưng đều chia sẻ cùng một tham số làm giảm đáng kể số lượng tham số trong mô hình. Một khi mạng nơ-ron tích chập (CNN) đã học cách nhận dạng một mẫu ở một vị trí, nó có thể nhận dạng mẫu đó ở bất kỳ vị trí nào khác. Ngược lại, một khi mạng nơ-ron kết nối đầy đủ (fully-connected neural network) đã học cách nhận dạng một mẫu ở một vị trí, nó chỉ có thể nhận dạng mẫu đó ở vị trí cụ thể đó.

Ảnh đầu vào cũng bao gồm nhiều lớp con: mỗi lớp tương ứng với một kênh màu. Thông thường có ba kênh: đỏ, xanh lá cây và xanh dương (RGB). Ảnh xám chỉ có một kênh, nhưng một số ảnh có thể có nhiều kênh hơn—ví dụ, ảnh vệ tinh thu thập thêm các tần số ánh sáng (như hồng ngoại).



Hình 14-6. Hai lớp tích chập với nhiều bộ lọc (kernel) trong mỗi lớp, xử lý ảnh màu có ba kênh màu. Mỗi lớp tích chập xuất ra một bản đồ đặc trưng cho mỗi bộ lọc.

Cụ thể, một nơ-ron nằm ở hàng i , cột j của bản đồ đặc trưng k trong một lớp tích chập l nhất định được kết nối với đầu ra của các nơ-ron trong lớp trước đó $l - 1$, nằm ở các hàng $i \times s$ đến $i \times s + f - 1$ và các cột $j \times s$ đến $j \times s + f - 1$, trên tất cả các bản đồ đặc trưng (trong lớp $l - 1$). Lưu ý rằng, trong cùng một lớp, tất cả các nơ-ron nằm ở cùng hàng i và cột j nhưng trong các bản đồ đặc trưng khác nhau đều được kết nối với đầu ra của chính các nơ-ron đó trong lớp trước đó.

Phương trình 14-1 tóm tắt các giải thích trước đó thành một phương trình toán học lớn: nó cho thấy cách tính toán đầu ra của một nơ-ron nhất định trong lớp tích chập. Nó hơi phức tạp do có nhiều chỉ số khác nhau, nhưng tất cả những gì nó làm là tính tổng có trọng số của tất cả các đầu vào, cộng với số hạng thiên vị.

Phương trình 14-1. Tính toán đầu ra của một nơron trong lớp tích chập.

$$z_{i,j,k} = b_{k'} + \sum_{u,v} w_{u,v,k'} \times x_{i',j',k'} \quad \text{với } \begin{cases} i' = i \times s_h + u \\ j' = j \times s_w + v \end{cases}$$

Trong phương trình này:

- $z_{i,j,k}$ là đầu ra của nơron nằm ở hàng i , cột j trong bản đồ đặc trưng k của lớp tích chập (lớp $z_{i,j,k}$).
- $b_{k'}$ Như đã giải thích ở trên, s_h và s_w là bước nhảy dọc và ngang, f và f là chiều cao và chiều rộng của trường tiếp nhận, và f là số lượng bản đồ đặc trưng trong lớp trước đó (lớp $l - 1$).
- $x_{i',j',k'}$ là đầu ra của nơron nằm ở lớp $l - 1$, hàng i' , cột j' , bản đồ đặc trưng k' (hoặc kênh k' nếu i', j', k') (Lớp trước đó là lớp đầu vào).
- $w_{u,v,k'}$ là hệ số thiên vị cho bản đồ đặc trưng k (trong lớp l). Bạn có thể coi nó như một núm điều chỉnh độ sáng tổng thể của bản đồ đặc trưng k .
- u, v là trọng số kết nối giữa bất kỳ nơron nào trong bản đồ đặc trưng k của lớp l và đầu vào của nó nằm ở u, v, k', k hàng u , cột v (tương đối so với trường tiếp nhận của tế bào thần kinh) và bản đồ đặc trưng k' .

Hãy cùng xem cách tạo và sử dụng lớp tích chập bằng Keras.

Triển khai các lớp tích chập với Keras

Đầu tiên, chúng ta hãy tải và tiền xử lý một vài hình ảnh mẫu, sử dụng hàm `load_sample_image()` của Scikit-Learn và các lớp `CenterCrop` và `Rescaling` của Keras (tất cả đều đã được giới thiệu trong [Chương 13](#)):

```
from sklearn.datasets import load_sample_images
import tensorflow as tf

images = load_sample_images()["images"]
images = tf.keras.layers.CenterCrop(height=70, width=120)(images)
images = tf.keras.layers.Rescaling(scale=1 / 255)(images)
```

Hãy cùng xem xét hình dạng của tensor hình ảnh:

```
>>> images.shape
TensorShape([2, 70, 120, 3])
```

Ồi trời, đây là tensor 4 chiều, chúng ta chưa từng thấy cái này bao giờ! Tất cả các chiều này có nghĩa là gì? Chà, có hai hình ảnh mẫu, điều này giải thích chiều thứ nhất. Sau đó, mỗi hình ảnh có kích thước 70×120 , vì đó là kích thước chúng ta đã chỉ định khi tạo lớp `CenterCrop` (hình ảnh gốc có kích thước 427×640). Điều này giải thích chiều thứ hai và thứ ba. Và cuối cùng, mỗi pixel chứa một giá trị cho mỗi kênh màu, và có ba kênh màu—đỏ, xanh lá cây và xanh lam—điều này giải thích chiều cuối cùng.

Bây giờ chúng ta hãy tạo một lớp tích chập 2D và đưa những hình ảnh này vào để xem kết quả. Để làm điều này, Keras cung cấp lớp `Convolution2D`, hay còn gọi là `Conv2D`. Về cơ bản, lớp này dựa vào phép toán `tf.nn.conv2d()` của TensorFlow. Hãy tạo một lớp tích chập với 32 bộ lọc, mỗi bộ lọc có kích thước 7×7 (sử dụng `kernel_size=7`, tương đương với việc sử dụng `kernel_size=(7, 7)`), và hãy áp dụng lớp này cho một lô nhỏ gồm hai hình ảnh của chúng ta:

```
conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7)
fmaps = conv_layer(images)
```

GHI CHÚ

Khi nói về lớp tích chập 2D, "2D" đề cập đến số chiều không gian (chiều cao và chiều rộng), nhưng như bạn thấy, lớp này nhận đầu vào 4D: như chúng ta đã thấy, hai chiều bổ sung là kích thước lô (chiều đầu tiên) và số kênh (chiều cuối cùng).

Bây giờ chúng ta hãy xem xét hình dạng của kết quả đầu ra:

```
>>> fmaps.shape
TensorShape([2, 64, 114, 32])
```

Dạng tín hiệu đầu ra tương tự như dạng tín hiệu đầu vào, với hai điểm khác biệt chính: thứ nhất, có 32 kênh thay vì 3 kênh. Điều này là do chúng ta đặt filters=32, nên ta nhận được 32 bản đồ đặc trưng đầu ra: thay vì cường độ của màu đỏ, xanh lá cây và xanh lam tại mỗi vị trí, giờ đây ta có cường độ của từng đặc trưng tại mỗi vị trí. Thứ hai, chiều cao và chiều rộng đều bị thu nhỏ đi 6 pixel. Điều này là do lớp Conv2D không sử dụng bất kỳ phần đệm bằng 0 nào theo mặc định, có nghĩa là chúng ta mất một vài pixel ở hai bên của bản đồ đặc trưng đầu ra, tùy thuộc vào kích thước của các bộ lọc. Trong trường hợp này, vì kích thước kernel là 7, chúng ta mất 6 pixel theo chiều ngang và 6 pixel theo chiều dọc (tức là 3 pixel ở mỗi bên).

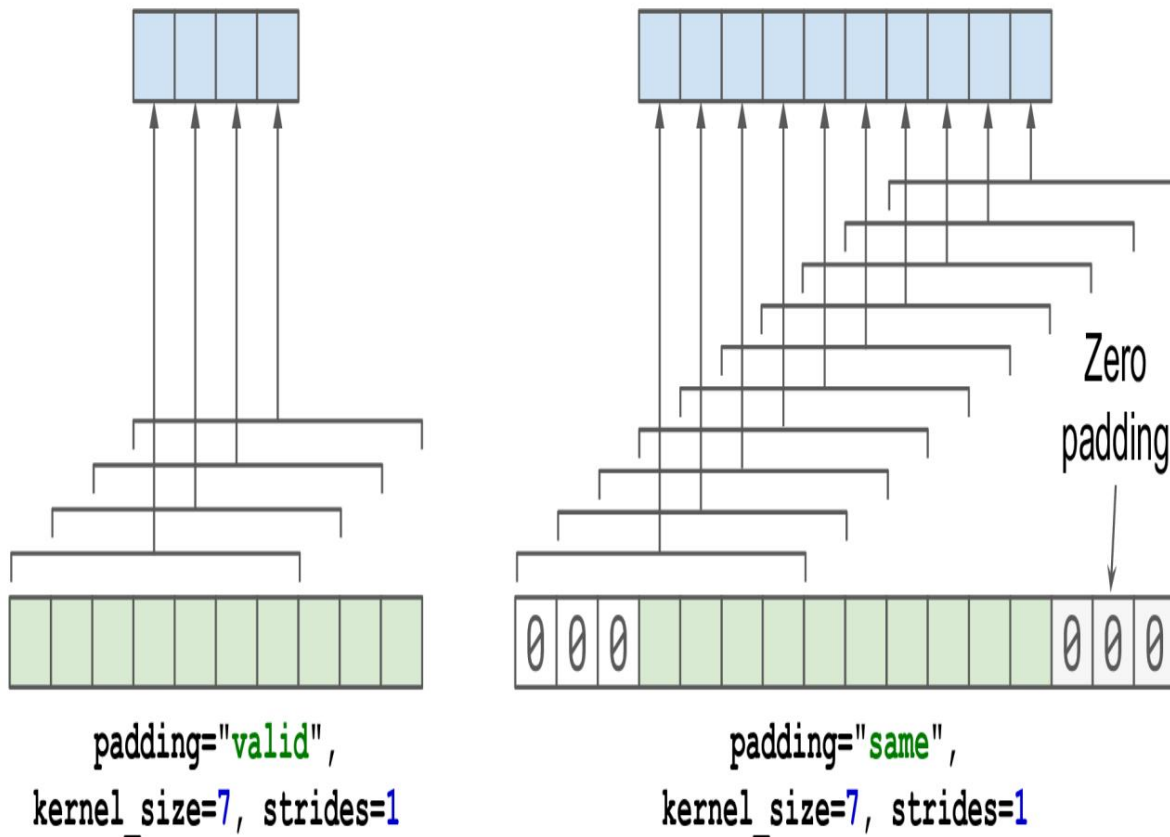
CẢNH BÁO

Tùy chọn mặc định có tên gọi khá bất ngờ là padding="valid", thực chất có nghĩa là không có phần đệm bằng số 0 nào cả! Tên gọi này xuất phát từ thực tế là trong trường hợp này, trường tiếp nhận của mỗi nơ-ron nằm hoàn toàn trong các vị trí hợp lệ bên trong đầu vào (nó không vượt ra ngoài giới hạn). Đây không phải là một đặc điểm đặt tên riêng của Keras: mọi người đều sử dụng cách đặt tên kỳ lạ này.

Nếu thay vào đó ta đặt padding="same", thì các đầu vào sẽ được thêm đủ số 0 ở tất cả các phía để đảm bảo rằng các bản đồ đặc trưng đầu ra có cùng kích thước với đầu vào (do đó có tên của tùy chọn này):

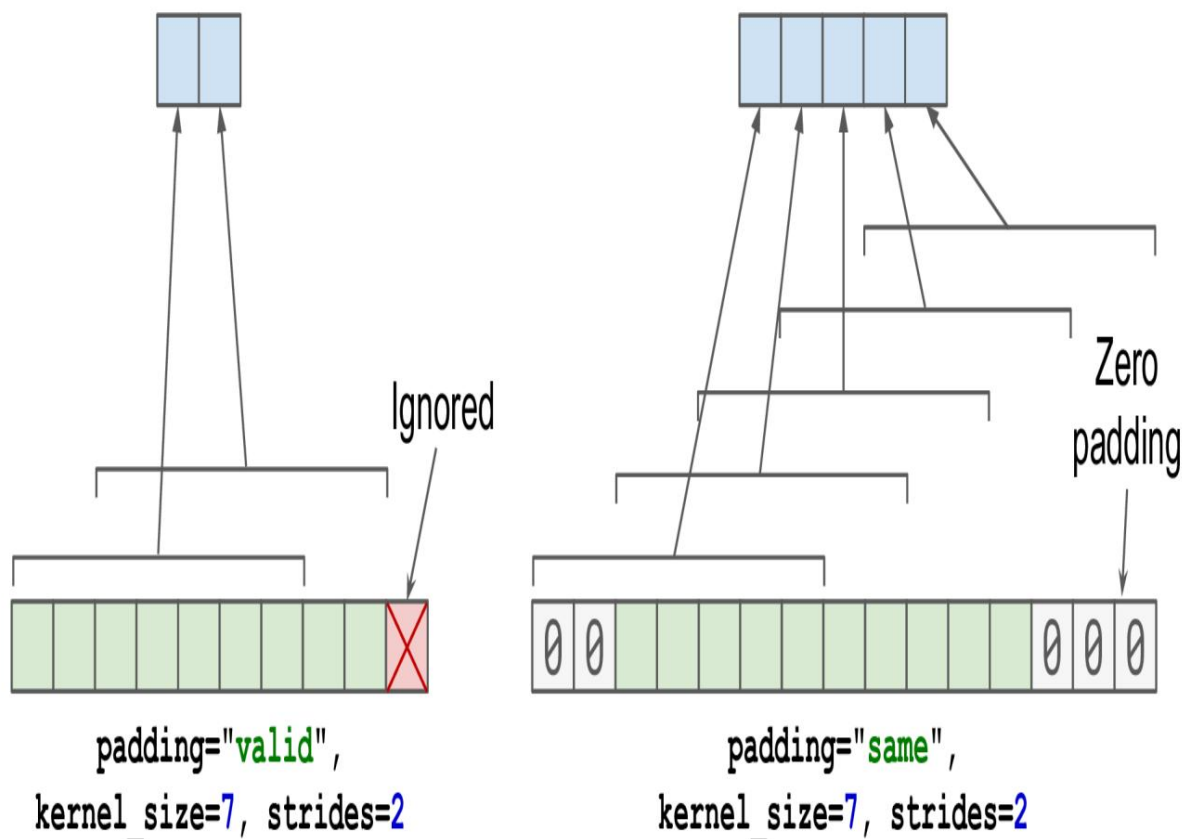
```
>>> conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7,
...                                     padding="same")
...
>>> fmaps = conv_layer(images)
>>> fmaps.shape
TensorShape([2, 70, 120, 32])
```

Hai tùy chọn khoảng đệm này được minh họa trong [Hình 14-7](#). Để đơn giản, ở đây chỉ hiển thị kích thước chiều ngang, nhưng tất nhiên logic tương tự cũng áp dụng cho kích thước chiều dọc.



Hình 14-7. Hai tùy chọn đệm khi strides=1

Nếu bước nhảy lớn hơn 1 (theo bất kỳ hướng nào), thì kích thước đầu ra sẽ không bằng kích thước đầu vào, ngay cả khi padding="same". Ví dụ, nếu bạn đặt strides=2 (hoặc tương đương strides=(2, 2)), thì bản đồ đặc trưng đầu ra sẽ là 35×60 : bị giảm một nửa cả theo chiều dọc và chiều ngang. Hình 14-8 cho thấy điều gì xảy ra khi strides=2, với cả hai tùy chọn padding.



Hình 14-8. Với bước nhảy lớn hơn 1, đầu ra nhỏ hơn nhiều ngay cả khi sử dụng đệm "giống nhau" (và đệm "hợp lệ" có thể bỏ qua một số đầu vào).

Nếu bạn tò mò, đây là cách tính kích thước đầu ra:

- Với padding="valid", nếu chiều rộng của đầu vào là i được làm tròn xuống, khi đó độ rộng đầu ra bằng $(i - f + s) / s$, làm tròn xuống. Nhớ rằng f là chiều rộng của kernel và s là bước nhảy ngang. Bất kỳ phần dư nào trong phép chia tương ứng với các cột bị bỏ qua ở phía bên phải của ảnh đầu vào. Có thể sử dụng cùng một logic để tính toán chiều cao đầu ra và bất kỳ hàng nào bị bỏ qua ở cuối hình ảnh.
- Với padding="same", chiều rộng đầu ra bằng i / s , số lượng cột làm tròn số. Để làm được điều này, thích hợp được thêm vào bên trái và bên phải của ảnh đầu vào (số lượng bằng nhau nếu có thể, hoặc chỉ thêm một hàng nữa ở phía bên phải). Giả sử chiều rộng đầu ra là o , sau đó là số lượng số 0 được thêm vào. Một lần nữa, logic tương tự có thể được sử dụng để tính toán chiều cao đầu ra, và $1) \times s + f - i$.

Bây giờ chúng ta hãy xem xét trọng số của lớp (được ký hiệu là w và b trong [Phương trình 14-1](#)). Giống như một lớp dày đặc, lớp này cũng có trọng số lớn.

k' , k layer, một lớp Conv2D chứa tất cả các trọng số của lớp, bao gồm cả kernel và bias. Các kernel được khởi tạo.

Ngẫu nhiên, trong khi các hệ số thiên vị được khởi tạo bằng 0. Các trọng số này có thể được truy cập dưới dạng biến TF thông qua các trọng số.

thuộc tính, hoặc dưới dạng mảng NumPy thông qua phương thức `get_weights()`:

```
>>> kernels, biases = conv_layer.get_weights()
>>> kernels.shape
(7, 7, 3, 32)
>>> biases.shape
(32,)
```

Mảng kernel có 4 chiều và hình dạng của nó là `[kernel_height, kernel_width, input_channels,`

`output_channels]`. Số lượng kênh đầu ra là

Bằng với số lượng bản đồ đặc trưng đầu ra, cũng bằng với số lượng bộ lọc.

Điều quan trọng nhất cần lưu ý là chiều cao và chiều rộng của ảnh đầu vào không xuất hiện trong hình dạng của kernel: điều này là do tất cả các neuron trong bản đồ đặc trưng đầu ra đều có cùng trọng số, như đã giải thích trước đó. Điều này có nghĩa là bạn có thể đưa ảnh có kích thước bất kỳ vào lớp này, miễn là chúng có kích thước ít nhất bằng kích thước của kernel và có đúng số kênh (3 trong trường hợp này).

Cuối cùng, bạn thường muốn chỉ định một hàm kích hoạt (như ReLU) khi tạo lớp Conv2D, và cũng chỉ định bộ khởi tạo kernel tương ứng (như khởi tạo He). Điều này là vì lý do tương tự như đối với các lớp Dense: một lớp tích chập thực hiện một phép toán tuyến tính, vì vậy nếu bạn xếp chồng nhiều lớp tích chập mà không có bất kỳ hàm kích hoạt nào, chúng sẽ tương đương với một lớp tích chập duy nhất, do đó chúng sẽ không thể học được bất cứ điều gì thực sự phức tạp.

Như bạn thấy, các lớp tích chập có khá nhiều siêu tham số: bộ lọc, kích thước kernel, đệm, bước nhảy, hàm kích hoạt, bộ khởi tạo kernel, v.v. Như mọi khi, bạn có thể sử dụng phương pháp kiểm định chéo để tìm ra các giá trị siêu tham số phù hợp, nhưng điều này rất tốn thời gian. Chúng ta sẽ thảo luận về các kiến trúc CNN phổ biến hơn ở phần sau của chương này, để giúp bạn hình dung được các giá trị siêu tham số nào hoạt động tốt nhất trong thực tế.

Yêu cầu bộ nhớ

Một thách thức khác với mạng CNN là các lớp tích chập yêu cầu một lượng RAM khổng lồ. Điều này đặc biệt đúng trong quá trình huấn luyện, bởi vì bước lan truyền ngược của thuật toán lan truyền ngược yêu cầu tất cả các giá trị trung gian được tính toán trong bước lan truyền thuận.

Ví dụ, hãy xem xét một lớp tích chập với 200 bộ lọc 5×5 , bước nhảy 1 và đệm "same". Nếu đầu vào là ảnh RGB 150×100 (ba kênh), thì số lượng tham số là $(5 \times 5 \times 3 + 1) \times 200 = 15.200$ (số + 1 tương ứng với các số hạng bias), khá nhỏ so với một lớp kết nối đầy đủ. Tuy nhiên, mỗi trong số 200 bản đồ đặc trưng chứa 150×100 nơ-ron, và mỗi nơ-ron này cần tính tổng có trọng số của $5 \times 5 \times 3 = 75$ đầu vào: đó là tổng cộng 225 triệu phép nhân số thực. Không tệ như một lớp kết nối đầy đủ, nhưng vẫn khá tốn kém về mặt tính toán. Hơn nữa, nếu các bản đồ đặc trưng được biểu diễn bằng số thực 32 bit, thì đầu ra của lớp tích chập sẽ chiếm $200 \times 150 \times 100 \times 32 = 96$ triệu bit (12 MB) RAM. Và đó chỉ là cho một trường hợp duy nhất – nếu một lô huấn luyện chứa 100 trường hợp, thì lớp này sẽ sử dụng tới 1,2 GB RAM!

Trong quá trình suy luận (tức là khi đưa ra dự đoán cho một trường hợp mới), bộ nhớ RAM mà một lớp chiếm dụng có thể được giải phóng ngay sau khi lớp tiếp theo được tính toán xong, vì vậy bạn chỉ cần lượng RAM bằng với lượng RAM cần thiết cho hai lớp liên tiếp. Nhưng trong quá trình huấn luyện, mọi thứ được tính toán trong quá trình truyền tiến cần được bảo toàn cho quá trình truyền ngược, vì vậy lượng RAM cần thiết (ít nhất) là tổng lượng RAM cần thiết cho tất cả các lớp.

MẸO

Nếu quá trình huấn luyện bị lỗi do hết bộ nhớ, bạn có thể thử giảm kích thước mini-batch. Ngoài ra, bạn có thể thử giảm chiều bằng cách sử dụng stride, hoặc loại bỏ một vài lớp. Hoặc bạn có thể thử sử dụng số thực 16 bit thay vì 32 bit. Hoặc bạn có thể phân phối mạng CNN trên nhiều thiết bị (chúng ta sẽ xem cách thực hiện điều này trong [Chương 19](#)).

Bây giờ chúng ta hãy xem xét khối cấu tạo phổ biến thứ hai của mạng CNN: lớp gộp (pooling layer).

Lớp gộp

Một khi bạn hiểu cách hoạt động của các lớp tích chập, các lớp gộp sẽ khá dễ nắm bắt. Mục tiêu của chúng là lấy mẫu con (tức là thu nhỏ) hình ảnh đầu vào để giảm tải tính toán, mức sử dụng bộ nhớ và số lượng tham số (nhờ đó hạn chế nguy cơ quá khớp).

Cũng giống như trong các lớp tích chập, mỗi nơ-ron trong lớp gộp được kết nối với đầu ra của một số lượng nơ-ron hạn chế trong lớp trước đó, nằm trong một vùng tiếp nhận hình chữ nhật nhỏ. Bạn phải xác định kích thước, bước nhảy và loại đệm, giống như trước đây. Tuy nhiên, một nơ-ron trong lớp gộp không có trọng số; tất cả những gì nó làm là tổng hợp các đầu vào bằng cách sử dụng một hàm tổng hợp như max hoặc mean. **Hình 14-9** hiển thị một lớp gộp tối đa, là lớp gộp phổ biến nhất.